POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Relazione delle attività laboratoriali per il corso di Fluidodinamica Sperimentale

Laboratorio 3: PARTICLE IMAGE VELOCIMETRY

Laurea Magistrale in Aeronautical Engineering - Ingegneria Aeronautica

Autori: Federico Poleni, Alberto Vacca, Giacomo Visconti, Marco Zappoli

Docente: Alex Zanotti

Assistenti: Donato Grassi, Luca Riccobene

Anno accademico: 2024-2025

Simboli utilizzati

I_i Sottodominio dell'immagine i, con $i = A, B$ ..... [-]

$\bar{I}_i$ Valore medio di luminosità di I_i, con $i = A, B$ ..... [-]

Δt Intervallo di tempo tra l'acquisizione di due frame consecutivi ..... $[s]$

$\Delta \bar{x}$ Spostamento medio delle particelle, atteso in un intervallo di tempo Δt ..... [px]

ϵ Scostamento del picco di correlazione ottenuto con interpolazione *subpixel* ..... [-]

ε *Offset* degli spostamenti per migliorare la robustezza del *median test* ..... [px]

ϕ Matrice che rappresenta il piano di correlazione tra I_A e I_B ..... [-]

E Energia cinetica del flusso medio ..... $[\mathrm{m}^2/\mathrm{s}^2]$

h Altezza della sezione di prova ..... [m]

k Energia cinetica turbolenta ..... $[\mathrm{m}^2/\mathrm{s}^2]$

l Lunghezza della camera di prova ..... [m]

M Fattore di magnificazione ..... [px/mm]

p N° di righe di cui viene effettuato l'*offset* di I_A su I_B, rispetto al centro di I_B ..... [-]

q N° di colonne di cui viene effettuato l'*offset* di I_A su I_B, rispetto al centro di I_B ..... [-]

r Residuo del *median test*, con r_c valore di soglia ..... [-]

r_i Mediana della quantità $|U_i - U_m|$ ..... [m/s]

U_{gall} Velocità nominale di prova in galleria ..... [m/s]

U_i Vettore velocità nell'i-esimo punto del campo di moto ..... [m/s]

U_m Mediana del vettore velocità nell'intorno di un punto ..... [m/s]

w Larghezza della sezione di prova ..... [m]

1. Introduzione

1.1. Cenni sulla PIV

Con *particle image velocimetry* si intende una famiglia di tecniche sperimentali basate sull'inseminazione della corrente mediante particelle e l'illuminazione della porzione di interesse del campo di moto mediante un fascio laser bidimensionale; il fine di queste operazioni è l'ottenimento tramite sensori ottici di coppie di immagini che consentano, mediante metodi di correlazione statistica, di determinare lo spostamento delle particelle illuminate tra due scatti consecutivi, informazione da cui stimare la velocità delle stesse. La natura non puntuale di tale misura, unita ad una risoluzione spazio-temporale confrontabile con quella di tecniche *hot-film anemometry* e ad una scarsa intrusività, rispondono all'esigenza di stimare il campo vettoriale della velocità in camera di prova con un'accuratezza tale da permettere valutazioni anche su grandezze derivate, come la vorticità. Le varianti tradizionali della PIV stanno ormai penetrando l'ambito industriale, mentre le evoluzioni più recenti estendono il loro concetto a misurazioni in un dominio tri-dimensionale (*scanning-light-sheet* e *tomographic* PIV), in flussi instazionari (*time-resolved* PIV) e in ambiti in cui la complessità delle misure è tipicamente elevata (*long-range microscopic* PIV per microfluidica e misurazioni in strati limite).
Il laboratorio sperimentale qui presentato è incentrato sulla PIV classica.

1.2. *Setup* sperimentale

1.2.1 Galleria del vento

Le attività descritte nel seguito sono state condotte presso il Laboratorio Sperimentale di Aerodinamica, nell'ambito del corso di Fluidodinamica Sperimentale.
L'impianto messo a disposizione è la galleria intitolata a Sergio De Ponte, una galleria a circuito chiuso di impostazione classica, dotata di una camera di prova avente sezione trasversale pari a $w = 1\ m \times h = 1.5\ m$ e lunga $l = 3\ m$. L'accesso ottico alla sezione di prova è quasi del tutto non ostruito, predisponendo l'impianto all'utilizzo dell'equipaggiamento per prove PIV.

1.2.2 Modello

L'oggetto di prova è la disposizione *side-by-side* di due rotori (diametro 30 cm) che rappresentano in scala 1:5 la propulsione installata sulla semiala di un velivolo eVTOL, con l'intenzione di studiarne la reciproca influenza in condizioni di *vortex ring state* (VRS); coerentemente, i rotori sono installati in modo tale che la spinta prodotta sia rivolta verso il divergente a valle della camera di prova, in direzione opposta a quella di provenienza della corrente a simulare un velivolo in fase di discesa verticale. La velocità di rotazione dei *propeller* è stata impostata a 7000 rpm, rispettando la similitudine in termini di numero di Mach rispetto al caso reale.

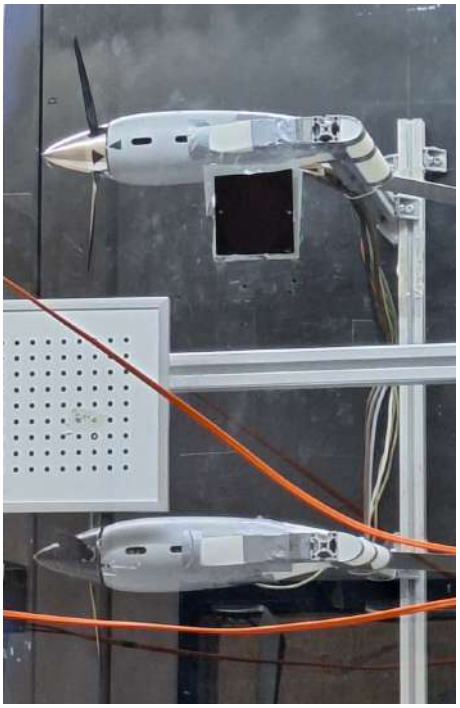


Figura 1: Coppia di rotori disposti *side-by-side* utilizzati per l'esperimento

1.2.3 Strumentazione e acquisizione

Il laser utilizzato per effettuare la PIV, di tipo a doppio impulso, è stato posizionato appena sotto la camera di prova. La frequenza massima con cui questo laser riesce a emettere il doppio impulso è 15 Hz, per cui non è stato possibile effettuare una *time-resolved* PIV. Nella parete inferiore della camera di prova è presente una finestra di plexiglass per permettere al laser di

illuminare il piano su cui si vuole ottenere la visualizzazione del campo di moto: tale piano è parallelo alle pareti laterali della camera di prova ed è situato a 6 *cm* dalla mezzeria dei rotori, per evitare che il propeller più in basso generi ombra e impedisca di osservare le particelle inseminanti.
Le fotocamere utilizzate per l'*imaging* sono due ILA.PIV.sCMOS, prodotte da ILA_5150 GmbH e dotate ciascuna di un sensore della tipologia *scientific*-CMOS che consente un'acquisizione di immagini 50 fps @ 5.5 Mpx [1]. Il posizionamento delle due fotocamere non è effettuato per una misura stereografica (per ottenere quindi tutte e tre le componenti di velocità), ma per poter rilevare un'area più ampia del piano di illuminazione. In Figura 2 è mostrato il *setup* utilizzato.
Per la gestione delle tempistiche di accensione del laser e apertura dell'otturatore delle fotocamere è presente un sincronizzatore a 5 canali, di cui 4 collegati al laser e 1, con un raccordo a T per sdoppiare il segnale, alle fotocamere. Per ottenere i dati per una prova PIV, ogni fotocamera deve scattare due fotogrammi, ognuno illuminato da un fascio laser; pertanto, i 4 canali collegati al laser corrispondono a 2 per ogni fascio. Il primo canale serve per inviare il segnale alla lampada laser, il secondo per il segnale al *Q-switch*. Regolando l'intervallo di tempo tra questi due segnali, è possibile regolare l'energia rilasciata dall'impulso di un singolo fascio laser; questa correlazione è data dal produttore del laser, di solito in forma grafica. In generale, diminuendo questo intervallo di tempo, aumenta l'energia rilasciata da ogni impulso del laser.

1.3. Acquisizione

Dopo aver acceso le ventole per far muovere l'aria nella galleria, il circuito è stato inseminato con DEHS tramite un serbatoio collegato all'aria compressa (atomizzatore a ugelli di Laskin); le particelle sono state iniettate a valle dei motori. L'intervallo di tempo tra l'invio del segnale alla lampada e il segnale al *Q-switch* è stato impostato pari a 200 μs, che permette di rilasciare un'energia di circa 150 mJ. L'intervallo tra i due impulsi laser, ovvero tra i due fotogrammi, è stato regolato uguale a 100 μs (nella Sezione 2 è spiegato il motivo di tale scelta), mentre il tempo di esposizione di ogni fotogramma è pari a 300 μs.

Figura 2: *Setup* sperimentale.

Il tempismo dei fotogrammi è stato sincronizzato con la posizione del rotore più in basso, per fare in modo che le pale di tale rotore risultino sempre nella stessa posizione in tutti i *frame*.

Sono state effettuate due acquisizioni distinte a due velocità di galleria diverse: una a 10 m/s, l'altra a 12 m/s; questi valori di velocità sono nell'intorno della velocità stimata per avere la condizione di VRS. Per ogni acquisizione, sono state scattate da entrambe le fotocamere 100 coppie di immagini, dove in ogni coppia la prima e la seconda immagine sono chiamate rispettivamente immagine A e B.

1.4. Obiettivi

Obiettivo principale del lavoro presentato nel seguito è quello di sviluppare uno *script* per il *processing* e l'analisi di dati *raw* acquisiti con la strumentazione per la PIV descritta nella precedente Sezione 1.2. Un'attenzione particolare verrà posta nei confronti delle strategie per eseguire la cross-correlazione delle immagini, a cui è dedicata l'intera Sezione 3.2, e alla reiezione de-

gli *outlier* presenti nel campo di velocità ottenuto, operazione fondamentale del *post-processing* (Sezione 3.3).

2. Calibrazione

Per ottenere risultati fisicamente interpretabili, è stato preliminarmente necessario calibrare la strumentazione utilizzata durante la prova sperimentale. L'obiettivo della calibrazione consiste nell'ottenere il fattore di magnificazione delle camere, necessario per convertire i valori misurati in pixel a valori in millimetri. Per fare ciò, in corrispondenza del piano illuminato dalla lama di luce del laser, è stato posizionato un *target* di calibrazione, e successivamente è stata acquisita un'immagine separata per ognuna delle due camere. Nella Figura 3 è rappresentata l'immagine originale ottenuta mediante l'affiancamento delle due immagini di calibrazione acquisite.

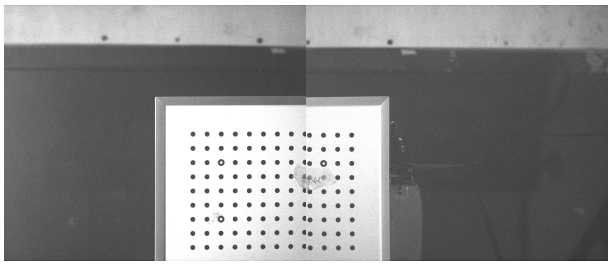


Figura 3: Immagine rappresentate il *target* di calibrazione, ottenuta affiancando l'acquisizione delle due telecamere.

Nota la distanza tra i punti del *target* di calibrazione, il fattore di magnificazione M è stato calcolato come il rapporto tra la distanza espressa in pixel tra i due punti e la loro distanza nota in millimetri:

$$M = \frac{119\ px}{17.57\ mm} = 6.7729 \left[\frac{px}{mm}\right] \qquad (1)$$

Una volta determinato questo valore, è stato individuato l'intervallo temporale ottimale da interporre tra la prima e la seconda immagine, al fine di ottenere uno spostamento medio $\Delta\bar{x}$ delle particelle traccianti pari a 8 pixel. Questo valore dello spostamento medio è stato scelto seguendo la regola dell'1/4, assumendo l'utilizzo di sottofinestre di cross-correlazione di dimensione 32x32 pixel.

$$\Delta t = \frac{\Delta\bar{x}}{U_{gall} \cdot M} \approx 100\ \mu s \qquad (2)$$

Il risulato dell'Equazione (2) rappresenta una media dei valori di Δt ottenuti utilizzando per U_{gall} le velocità di galleria usate durante le prove sperimentali (ovvero 10 e 12 m/s).
Infine, dopo aver sincronizzato le telecamere e i laser in base dell'intervallo temporale scelto, è stata acquisita una sequenza di 100 coppie di immagini per telecamera.

3. Analisi delle immagini

Per l'elaborazione delle immagini acquisite in laboratorio, il gruppo ha sviluppato un algoritmo implementato in MATLAB® in grado di eseguire l'analisi di cross-correlazione necessaria per il trattamento e l'interpretazione dei dati. In questa sezione vengono descritti la struttura del codice, modelli di analisi implementati e i risultati ottenuti. L'algoritmo si suddivide in tre macro-moduli principali:

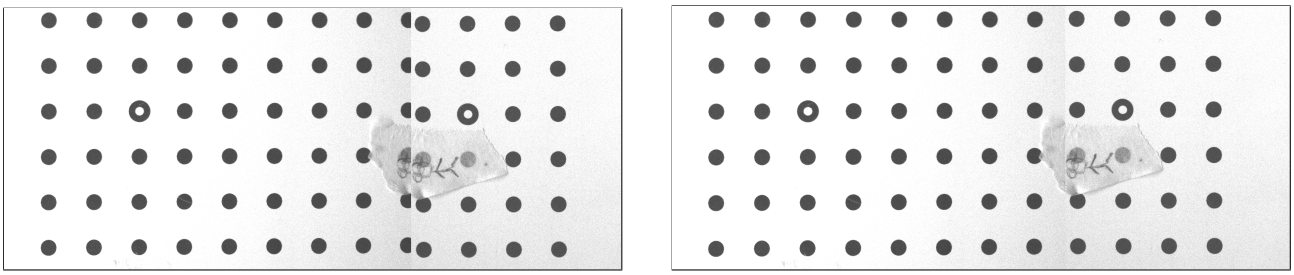
1. *pre−processing* delle immagini;
2. analisi statistica di cross-correlazione per ciascuna coppia di immagini;
3. *post−processing* dei risultati ottenuti.

3.1. *Pre-processing*

Come mostrato dalla Figura 3, le due telecamere presentano una zona di sovrapposizione dell'inquadratura e risultano leggermente disallineate tra loro. Pertanto, il primo passo è stato rimuovere i pixel sovrapposti e riallineare le due immagini, al fine di eseguire correttamente le analisi PIV successive. Nella Figura 4 viene mostrato un dettaglio del *target* di calibrazione nella zona di sovrapposizione, prima e dopo il processo descritto sopra.
Inoltre è stato applicato un filtro alle immagini per uniformare la luminosità dello sfondo e migliorare il contrasto, utilizzando il comando `imflatfield`. Quest'ultimo applica una correzione di *flat−field* utilizzando uno *smoothing* Gaussiano. Infine, le aree non rilevanti per l'analisi del campo di moto, come le zone d'ombra, le *nacelle* dei rotori e la barra di supporto, sono state mascherate.

3.2. Cross-correlazione

Questa parte dell'algoritmo rappresenta l'elemento cruciale per garantire un'esecuzione ottimizzata ed efficace del codice finalizzato all'analisi PIV. Si sono sviluppati gli algoritmi sia



(a) (b)

Figura 4: Dettaglio del *target* di calibrazione, prima (a) e dopo (b) l'allineamento delle immagini.

per una correlazione standard, ottenuta tramite sommatoria estesa a tutti i pixel dell'immagine del prodotto dell'intensità luminosa, sia per la correlazione ottenuta grazie alla trasformata di Fourier.
In entrambi i casi sono stati utilizzati sottodomini grandi 32 $px \times 32$ px; due sottodomini consecutivi sono sovrapposti per metà delle loro dimensioni, ovvero 16 px (sovrapposizione corrispondente ad un fattore di *overlap* tipico del 50%), al fine di incrementare il numero di campionamenti e migliorare l'accuratezza del calcolo. Dal momento che le dimensioni del sensore sono 2560 $px \times 2160$ px, il numero totale di punti in cui è necessario calcolare la velocità tramite correlazione è $316 \times 134 = 42344$.
Per ogni sottodominio, prima di calcolare la correlazione è stato sottratto il valore medio di luminosità del sottodominio dal sottodominio stesso. Questa sottrazione è stata effettuata indipendentemente per i sottodomini corrispondenti delle immagini A e B. Per evitare che dopo tale operazione valori negativi moltiplicati tra loro diano contributo positivo nella sommatoria per il calcolo della cross-correlazione, i valori negativi sono stati posti uguali a zero.

3.2.1 Calcolo standard attraverso sommatoria di prodotti

La matrice di correlazione viene calcolata attraverso la seguente formula:

$$\phi(p,q) = \sum_{i=1}^{32}\sum_{j=1}^{32} \left(I_A(i,j) - \bar{I}_A\right) \times \left(I_B(i+p, j+q) - \bar{I}_B\right) \qquad (3)$$

Nella Figura 5, un esempio di sottodomini delle immagini A e B, estratti da una porzione del campo inquadrato dalla Camera 1, appena sotto alla pala inferiore del rotore più in basso.

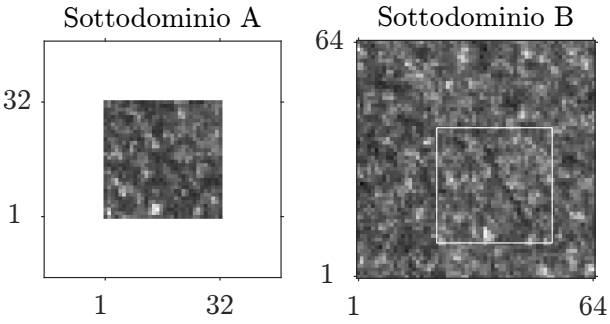


Figura 5: Esempio di sottodomini A e B.

Il rettangolo bianco nel sottodominio B corrisponde al contorno del sottodominio A traslato nel sottodominio B di una quantità che corrisponde alla posizione del picco del piano di correlazione, mostrato in Figura 6 (dove l'intensità luminosa è normalizzata rispetto al valore più alto).

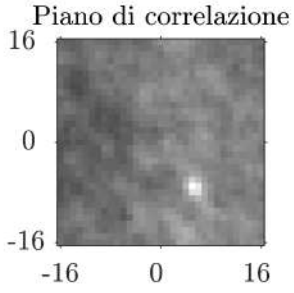


Figura 6: Piano di correlazione ottenuto dai sottodomini A e B della Figura 5.

Le coordinate del picco corrispondono allo spostamento più probabile, misurato in pixel, delle particelle in questo sottodominio.

3.2.2 Calcolo attraverso la trasformata di Fourier

In alternativa al metodo della sommatoria di prodotti, appena descritto, l'utilizzo opportuno della trasformata di Fourier discreta (DFT) per il calcolo della correlazione consente una notevole riduzione delle tempistiche richieste per l'operazione. All'interno del codice MATLAB®, riportato in Appendice B, questo viene fatto attraverso le funzioni `fft2` e `ifft2` per il calcolo della DFT bi-dimensionale, e della sua inversa, mediante algoritmo *Fast Fourier Transform* (FFT). In questo modo, il piano di correlazione si ottiene nel dominio di Fourier mediante il prodotto elemento per elemento delle matrici risultanti dall'applicazione di `fft2` ai sottodomini A e B; il piano di correlazione reale segue dall'utilizzo di `ifft2` sul piano rappresentato nel dominio di Fourier. Passaggio fondamentale per l'utilizzo corretto del codice nel caso dell'utilizzo della trasformata di Fourier è quello di riordinare le componenti della matrice ottenuta usando il comando `fft2`. In particolare, l'*output* di `fft2` dispone le basse frequenze nei quattro angoli della matrice, mentre le frequenze più alte si trovano nella parte centrale, rendendo difficile interpretare correttamente i dati. Per ovviare a ciò, si è utilizzato il comando `fftshift`, che riordina la matrice posizionando le frequenze basse al centro e quelle alte ai bordi. Questo riordinamento è fondamentale per ottenere una rappresentazione coerente del piano di correlazione nel contesto dello studio PIV, dove è essenziale che il dominio delle frequenze sia centrato sulla frequenza zero per una corretta interpretazione e analisi dei risultati.
È stato confrontato il tempo di esecuzione necessario per calcolare la matrice di correlazione per un singolo sottodominio con i due diversi metodi, ovvero il calcolo attraverso sommatoria di prodotti e il calcolo attraverso la trasformata di Fourier. Con il primo metodo, il tempo medio di riempimento della matrice di correlazione risulta essere $3.11 \cdot 10^{-3}$ s, mentre col secondo si ottiene un tempo medio di $1.35 \cdot 10^{-4}$ s, ovvero appena 1/23 rispetto all'altro metodo. Per essere sicuri che i tempi di esecuzione non siano troppo variabili tra diverse esecuzioni dello stesso script (in base alle risorse che il computer dedica a MATLAB® istante per istante), il codice è stato eseguito più volte e si è osservato che il tempo medio di esecuzione è pressoché costante.

3.2.3 Interpolazione *subpixel*

Inizialmente le componenti di spostamento (e quindi velocità) di ogni sottodominio sono state ottenute semplicemente individuando il picco del piano di correlazione come punto dove ϕ è massima.
Queste componenti di spostamento hanno un'incertezza elevata, ovvero $0.5\ px$, dal momento che corrispondono a valori interi di pixel. Pertanto, per migliorare l'accuratezza di questi risultati, è stata implementata una funzione per l'interpolazione *subpixel* attraverso un *fitting* gaussiano a tre punti. Con la seguente formula, è possibile ottenere per ognuna delle due direzioni (orizzontale e verticale) lo scostamento tra la posizione del picco ottenuto in precedenza e quello della funzione gaussiana che interpola il picco (ϕ_0) e i due valori adiacenti (ϕ_{-1} e ϕ_{+1}):

$$\epsilon = \frac{1}{2}\frac{ln\ \phi_{-1} - ln\ \phi_{+1}}{ln\ \phi_{-1} - 2ln\ \phi_0 + ln\ \phi_{+1}} \tag{4}$$

Nella Figura 7 è possibile osservare il miglioramento nell'individuazione del picco grazie a questa tecnica, per raggiungere un'accuratezza più elevata.

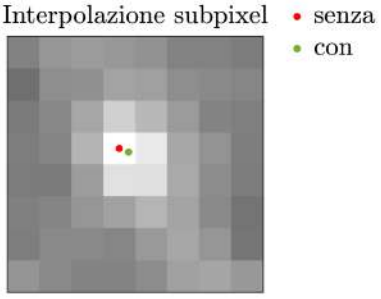


Figura 7: Zoom della Figura 6 intorno al picco di correlazione, per mostrare l'efficacia dell'interpolazione *subpixel*.

A questo punto, si risale allo spostamento vero e proprio, misurato in mm, grazie al fattore di magnificazione calcolato nell'equazione (1); dividendo quest'ultimo per Δt, si ottiene il valore di velocità corrispondente a quel sottodominio.

3.3. *Post-processing*

Infine, una volta generato il campo di moto tramite analisi statistica, il codice esegue diversi processi di *post−processing* per filtrare e ottimizzare i dati, rendendoli pronti per la visualizzazione. Nello specifico, vengono eseguite tre routine consecutive per il miglioramento dei dati:

1. ricalcolo del campo di moto nella regione di congiunzione delle due immagini;
2. interpolazione dei dati nella regione d'ombra generata dal passaggio della pala;
3. identificazione, eliminazione e successiva interpolazione dei vettori spuri, al fine di ridurre il rumore e migliorare l'accuratezza del campo vettoriale.

3.3.1 Regione di congiunzione

Questa regione si è dimostrata critica poiché le due telecamere, acquisendo valori di intensità luminosa significativamente diversi in quel punto, hanno introdotto errori nell'algoritmo di analisi del campo di moto, compromettendo l'accuratezza della stima in quella specifica area. Per correggere tale errore, per ciascuna coppia di immagini analizzata, è stato deciso di rimuovere 5 domini di dimensione 32 $px \times 32$ px per riga, centrati sulla sottofinestra che attraversa esattamente la congiunzione. Successivamente, i dati mancanti sono stati ricostruiti mediante un'interpolazione lineare eseguita riga per riga, utilizzando come nodi di interpolazione i valori immediatamente adiacenti a quelli rimossi.

3.3.2 Zona d'ombra generata dal passaggio della pala

Una procedura analoga è stata applicata per interpolare i dati nella regione d'ombra presente nel campo di moto. Durante l'analisi preliminare della prima coppia di immagini, è stata identificata con precisione la posizione occupata dall'ombra. Analogamente alla regione di congiunzione, sono stati rimossi i sottodomini di dimensione 32 $px \times 32$ px attraversati dall'ombra. Successivamente, per ciascuna coppia di immagini analizzate, i dati sono stati ricostruiti mediante interpolazione lineare, questa volta eseguita lungo le colonne.

3.3.3 Vettori spuri

Il metodo adottato per il rilevamento, la rimozione e l'interpolazione dei vettori spuri segue lo schema descritto in [2]. È stato adottato un approccio locale, piuttosto che globale, basato sulla valutazione della coerenza del campo di spostamenti. Questo metodo confronta ciascun dominio di dimensione 32 $px \times 32$ px con gli 8 domini che costituiscono l'intorno 3×3 del dominio analizzato. L'equazione caratteristica del *median test* è:

$$r = \frac{|\mathbf{U}_0 - \mathbf{U}_m|}{r_m + \varepsilon} \tag{5}$$

dove r rappresenta il residuo del vettore di spostamento $\mathbf{U}_0$, $\mathbf{U}_m$ è il vettore mediano degli 8 vettori nell'intorno di $\mathbf{U}_0$, r_m è la mediana di $r_i = |\mathbf{U}_i - \mathbf{U}_m|$ (con $i = 1 \dots 8$), e ε è un *offset* di 0.1 px introdotto per migliorare la robustezza del test, favorendo il rilevamento di piccole variazioni nei dati. Un vettore è considerato spurio se il suo residuo r supera una determinata soglia r_c. Seguendo le indicazioni fornite in [2], è stato scelto un valore di soglia $r_c = 2$. Eliminato il vettore spurio, il dato è stato interpolato attraverso una media pesata dei vettori presenti nell'intorno 3×3.

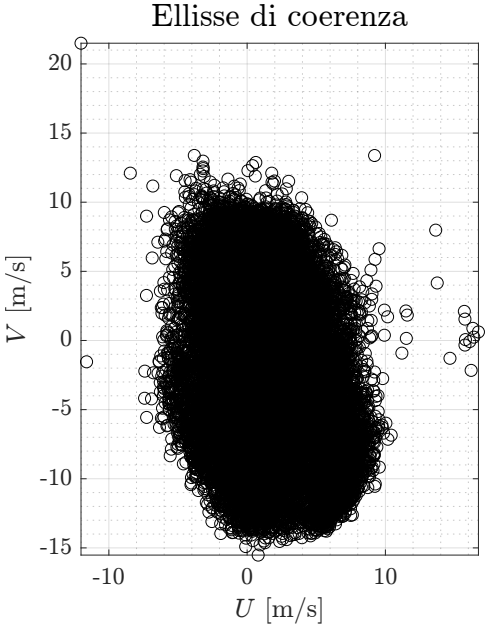


Figura 8: Esempio di ellisse di coerenza che mostra i vettori del campo di moto medio, ottenuto a partire da tutte e 100 le coppie di immagini acquisite per la prova a 10 m/s (Figura 9b).

3.4. Tentativo di implementazione della *Multi-pass interrogation*

Nel trascritto dello *script* in Appendice B è presente la possibilità di eseguire il codice con la tecnica di interrogazione avanzata denominata *window shift method* o *multiple pass interrogation*. Tuttavia, nella Sezione 4 non si riportano i relativi risultati in quanto sono state evidenziate notevoli deviazioni delle *streamline* e un aumento del modulo della velocità rispetto alla cross-correlazione standard, non compatibili con un semplice miglioramento della risoluzione che la tecnica si propone di offrire.
Le rispettive righe di codice vengono riportate per completezza, ma si rimanda a [2] per una trattazione completa della metodologia.

4. Analisi dei risultati

Prova sperimentale a 10 m/s

La prima prova eseguita, i cui risultati sono riportati di seguito, è stata effettuata utilizzando una velocità della galleria del vento pari a 10 m/s. Nella Figura 9 sono presentati i risultati, che contengono due informazioni principali: il colore dello sfondo rappresenta il modulo della velocità media calcolato su 100 coppie di immagini, mentre le linee di corrente illustrano il campo di velocità medio, fornendo una rappresentazione visiva della direzione e dell'intensità del flusso. Con riferimento ai risultati riportati in Appendice A (Figura 14), si può ritenere che la media su più immagini sia indispensabile per una buona risoluzione dei risultati.
Come mostrato nella Figura 9c, nelle regioni caratterizzate da una buona illuminazione, i risultati ottenuti mediante i due metodi si sovrappongono quasi perfettamente (errore relativo $err_{rel} = 5\%$). Tuttavia, nelle aree d'ombra generate dalle *nacelle* dei rotori, i risultati differiscono significativamente. Un'analisi più dettagliata delle Figure 9a e 9b evidenzia come il metodo basato sulla trasformata di Fourier restituisca una soluzione più fisicamente consistente nelle aree sotto i rotori, caratterizzate da *streamline* più regolari. Questo suggerisce una maggiore robustezza del metodo basato sella DFT in condizioni di rumore o scarsa qualità dell'immagine. Analizzando la Figura 9, si osservano tre strutture vorticose principali, correlate ai due tori formati dalla scia dei rotori. Nella Figura 10 sono mostrati i valori dell'energia cinetica associata al flusso medio (E) e dell'energia cinetica turbolenta (k).

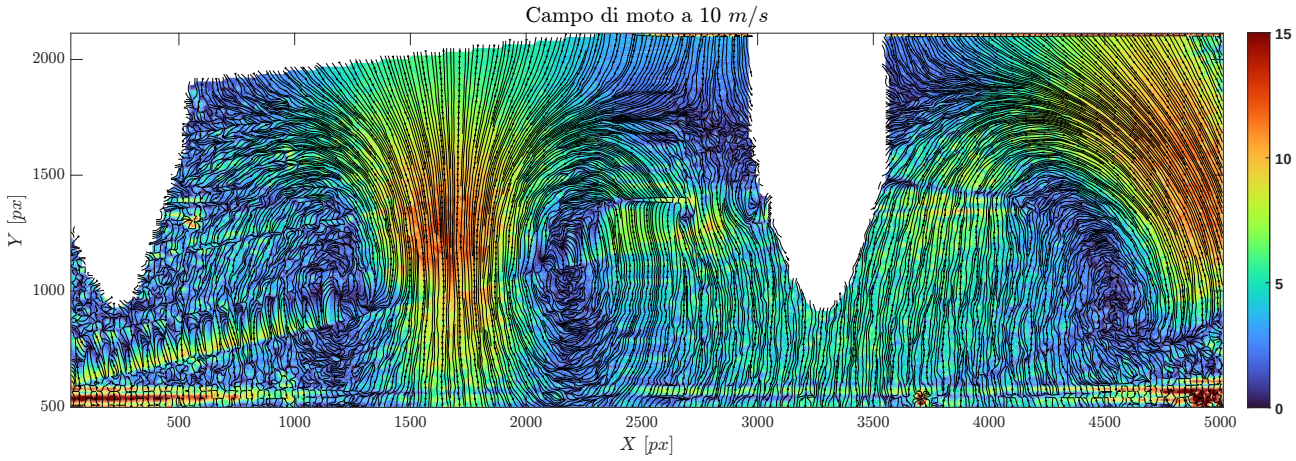
Prova sperimentale a 12 m/s

Analogamente a quanto fatto per la prova sperimentale a 10 m/s, lo stesso approccio è stato applicato alla prova sperimentale a 12 m/s. Nelle Figure 11a, 11b e 11c, rispettivamente, vengono presentati i risultati, ottenuti mediante l'utilizzo della DFT, relativi al campo di moto mediato sulle 100 immagini con le linee di corrente, l'energia cinetica associata al moto medio e l'energia cinetica turbolenta. Come atteso, il campo di moto medio evidenzia un'accelerazione dovuta alla maggiore velocità della galleria, mentre le strutture vorticose vengono trasportate dal moto medio più lontano dalle pale dei rotori.

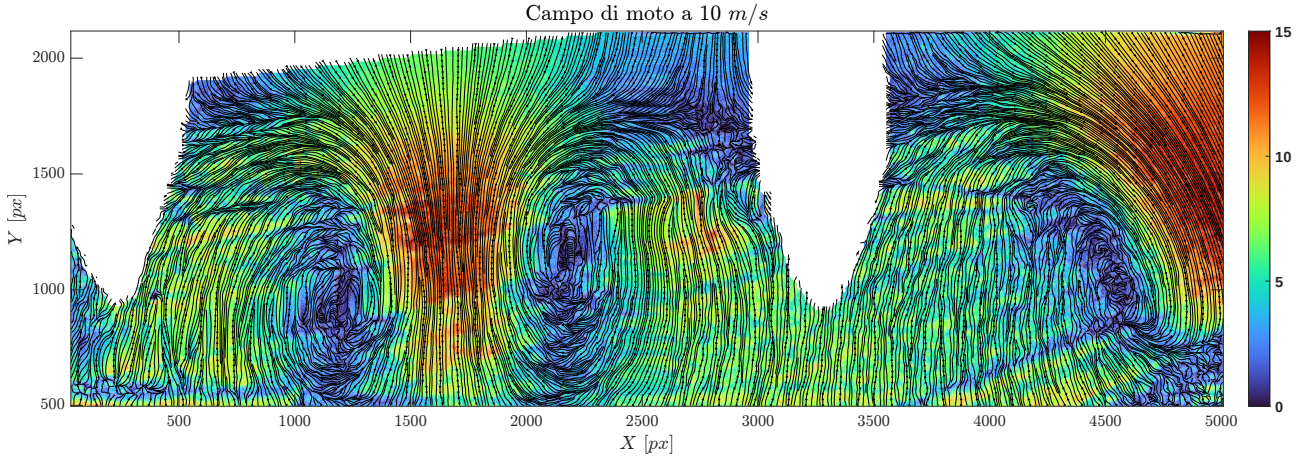
4.1. Confronto con PIVlab

Al fine di validare i risultati ottenuti nei paragrafi precedenti, è stato deciso di utilizzare un *software open-source* per effettuare un'analisi PIV. Fra i possibili è stato scelto PIVlab, un *toolbox* presente in MATLAB®. All'interno di esso sono presenti svariate tecniche finalizzate ad ottenere risultati il più accurati possibili, ma all'interno di questo studio si è deciso di utilizzare le stesse tecniche adottate nel paragrafo 3.2.2 nel caso con velocità di galleria pari a 10 m/s (interpolazione *subpixel* compresa). È bene sottolineare che, non tutti gli algoritmi e funzioni implementate nel codice del gruppo sono presenti all'interno di PIVlab, portando inevitabilmente a risultati diversi. Di seguito sono elencati le differenze sostanziali:

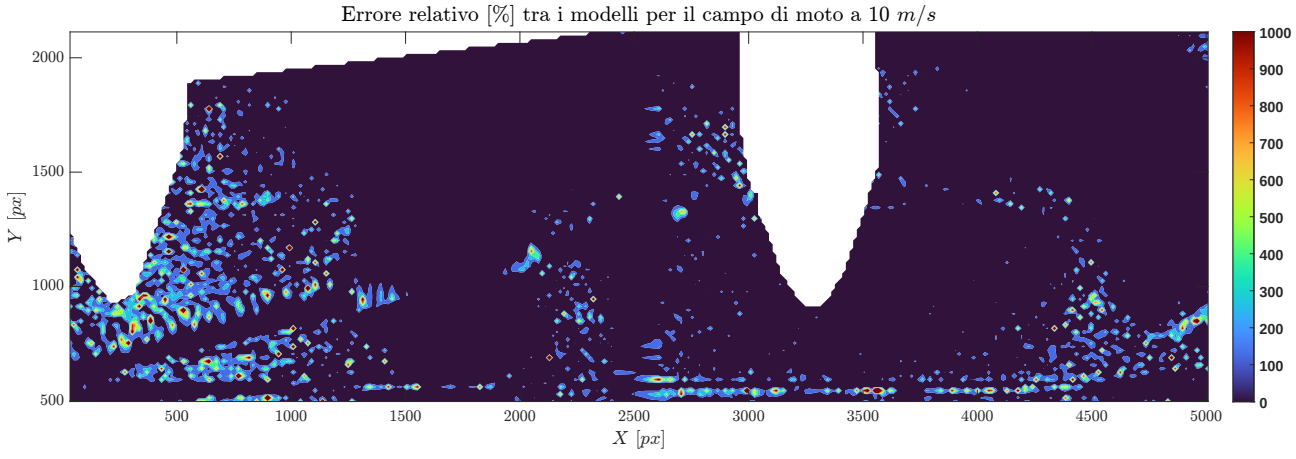
- non è stata utilizzata la funzione `imflatfield`. Difatti nessun algoritmo di *pre-processing* è stato adottato;
- in entrambe le analisi è stato impostato un limite sulla velocità; nel caso di PIVlab il limite viene impostato sulle singole componenti del vettore, mentre l'algoritmo sviluppato dal gruppo il limite viene impostato sul modulo;
- gli algoritmi di interpolazione dei vettori mancanti risultano diversi.



(a)

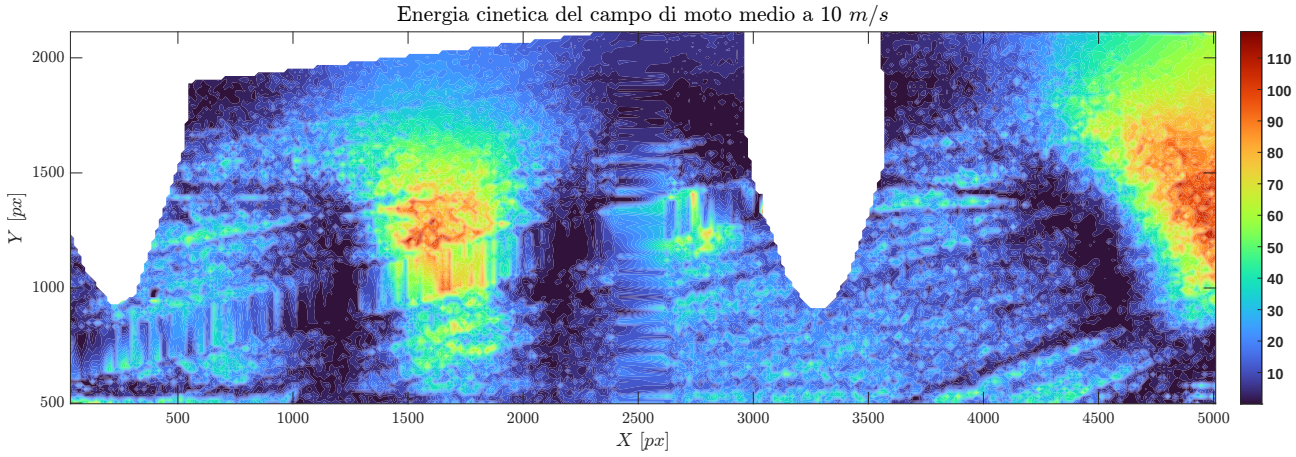


(b)

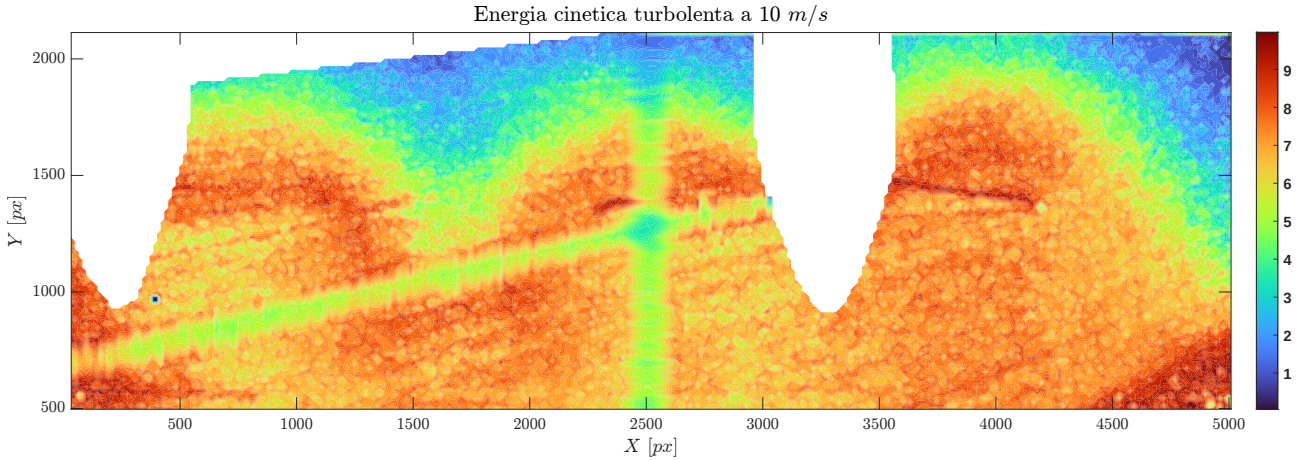


(c)

Figura 9: Risultato del campo di moto medio ottenuto mediante l'utilizzo della sommatoria dei prodotti (a) e la trasformata di Fourier (b) per la prova sperimentale a 10 m/s. L'errore percentuale tra le due analisi è presentato in (c).

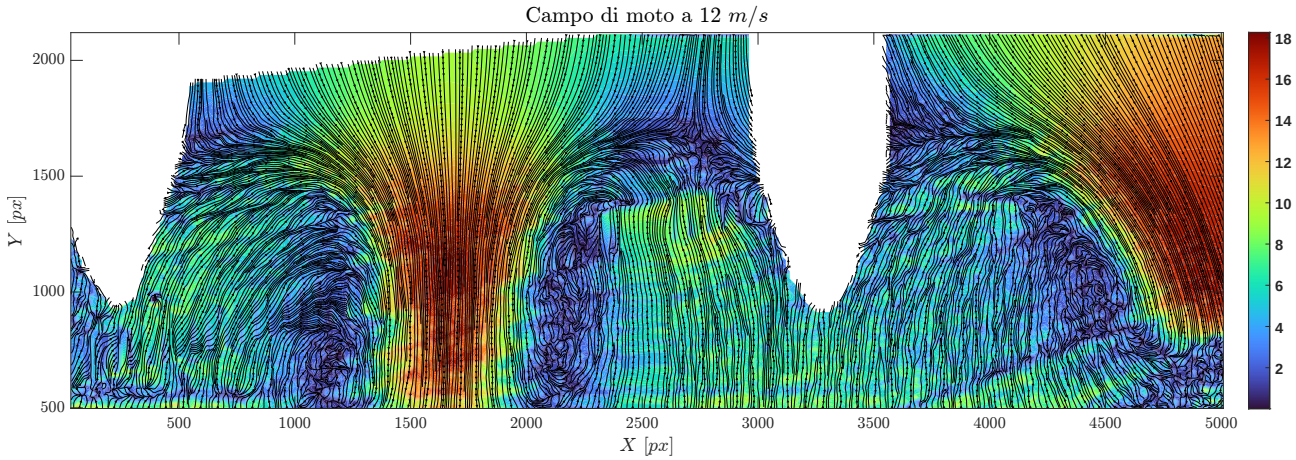


(a)

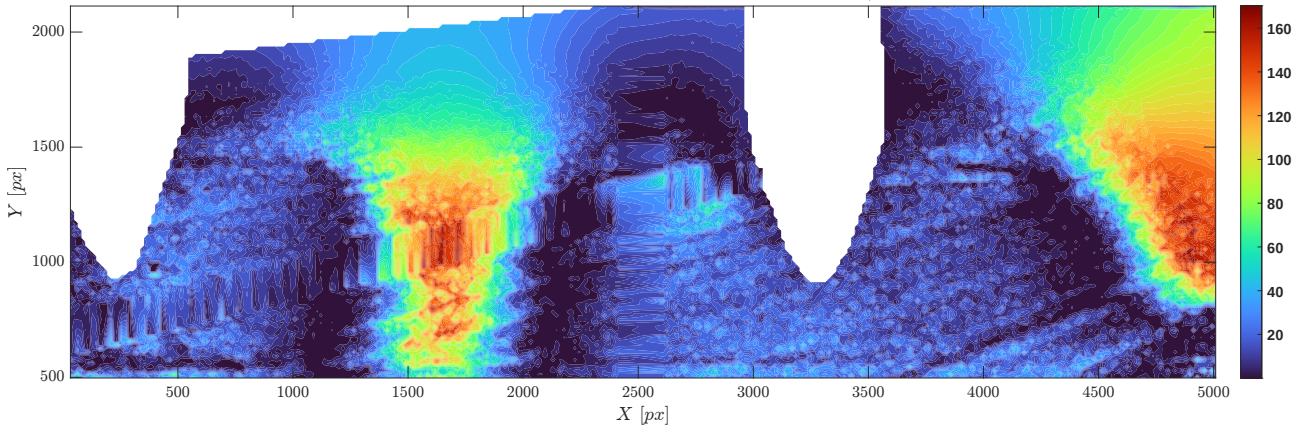


(b)

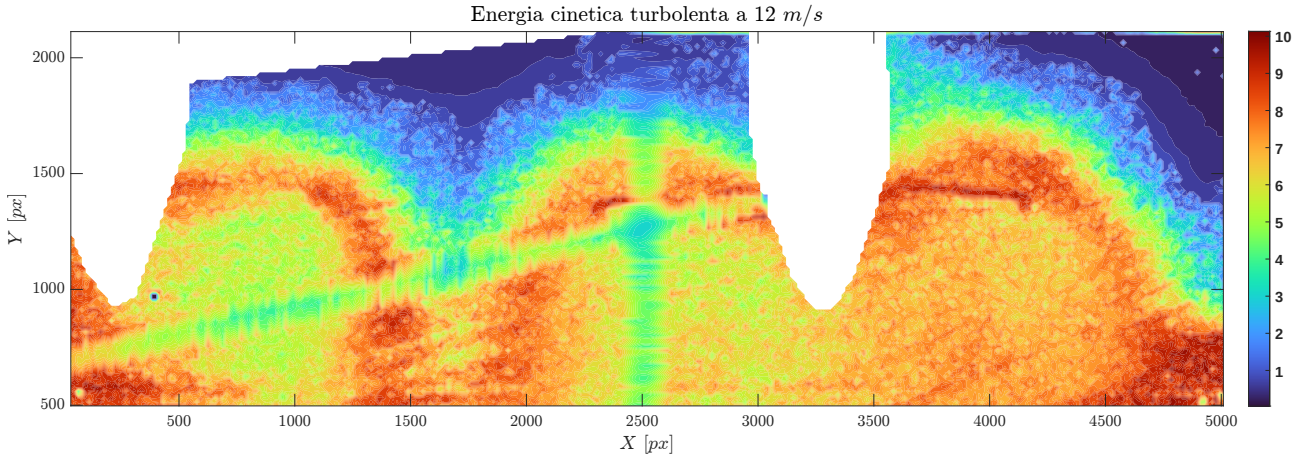
Figura 10: Energia cinetica associata al flusso medio (a) e energia cinetica turbolenta (b) per la prova sperimentale a 10 m/s. Il *pattern* a croce nel centro dell'immagine è un artefatto dato dall'algoritmo di interpolazione lineare adottato per la zona di congiunzione e per la zona d'ombra.



(a)

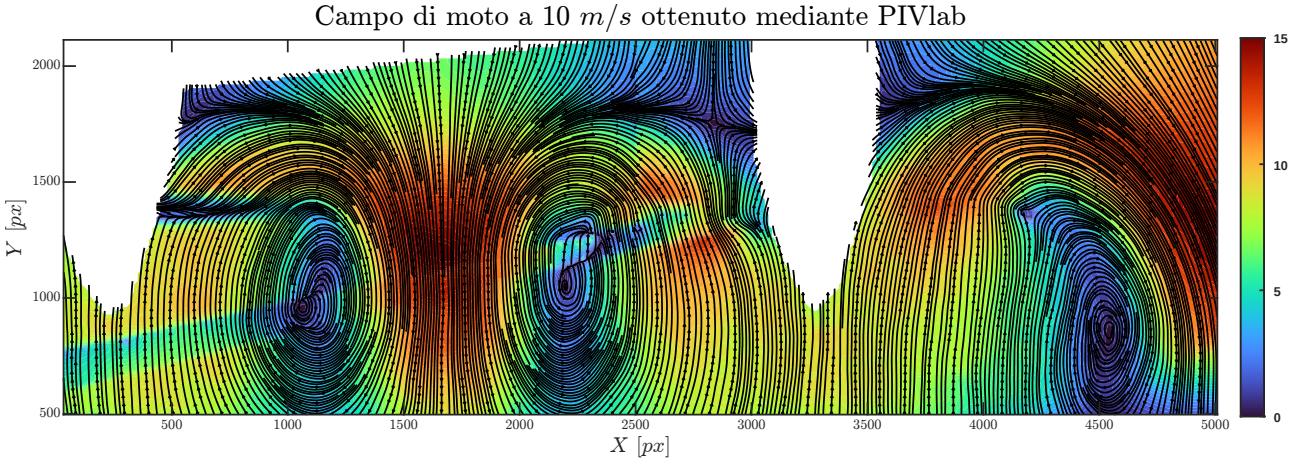


(b)

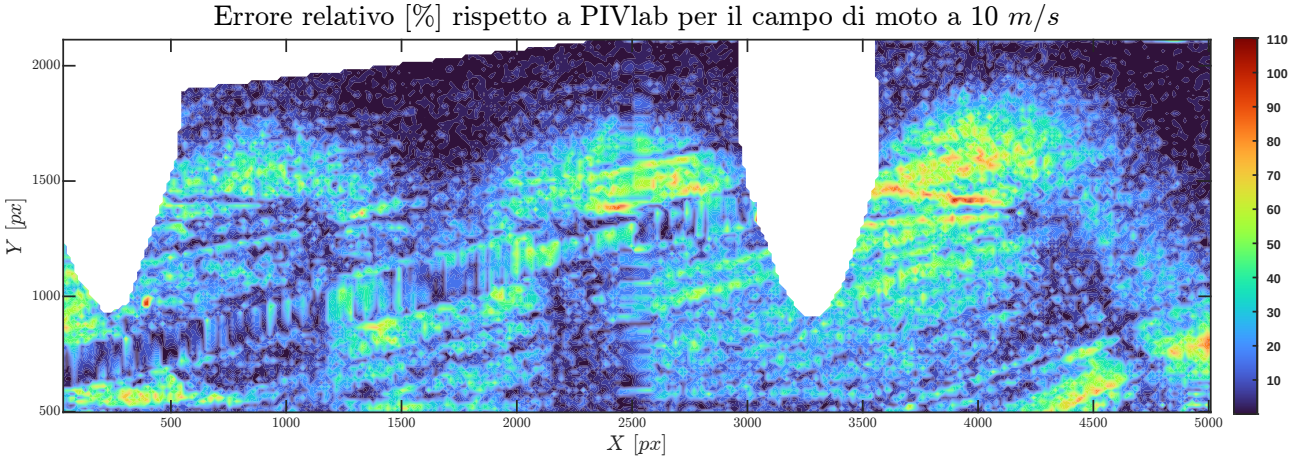


(c)

Figura 11: Risultati ottenuti mediante la trasformata di Fourier per la prova sperimentale a 12 m/s. Campo di moto medio (a), energia cinetica associata al flusso medio (b) e energia cinetica turbolenta (c).



(a)



(b)

Figura 12: (a) Risultato del campo di moto medio ottenuto mediante PIVlab con algoritmo di FFT per la prova sperimentale a 10 m/s. (b) L'errore percentuale tra l'analisi tramite PIVlab e risultato rappresentato in Figura 9b

In Figura 12a è riportato il grafico sul modulo della velocità ottenuto nel caso di PIVlab mediando le 100 coppie di immagini.
Per avere un confronto diretto con i risultati ottenuti dal gruppo si è deciso di calcolare la differenza dei moduli su ogni finestra di 32 $px \times 32$ px e normalizzare rispetto alla velocità asintotica (vedi Figura 12b). Si può notare che l'errore relativo risulta essere elevato nelle zone più critiche citate nei paragrafi precedenti: in particolare, si evidenzia la bassa efficienza dell'algoritmo in prossimità delle zone d'ombra e vicino alle *nacelle* all'altezza del rotore a causa del riflesso delle eliche. Invece, nelle zone illuminate omogeneamente l'errore scende al di sotto del 5%.

4.2. Interpretazione e limitazioni dei risultati

È chiaro che i risultati, soprattutto il campo scalare del modulo della velocità, siano affetti da errori dovuti alla natura disomogenea dell'illuminazione: in Figura 13, ottenuta mediando tutti i *frame* a disposizione, si osserva distintamente l'ombra di una delle pale del rotore inferiore, insieme ad un'alternanza di numerose bande chiare e scure di cui alcuni difetti sulla finestra trasparente in PMMA potrebbero essere responsabili; in ogni caso, tali fasci d'ombra hanno una rilevanza sufficiente da lasciare una propria traccia sulle immagini correlate, sotto forma di discontinuità nelle linee di corrente e nella mappa scalare delle componenti di velocità (si vedano, ad esempio, la Figura 9a e la 9b).
Visto l'elevato contrasto di questi chiaroscuri, nessuna tecnica di sottrazione del *background* ha consentito un netto miglioramento della correlazione.
L'illuminazione disomogenea, unitamente al rumore di fondo, hanno impedito di ottenere risultati significativi per quanto riguarda le grandezze derivate di rotore e divergenza del campo di velocità; l'operazione di derivazione numerica mostra, infatti, una notevole sensibilità a tale tipologia di errore e le funzioni MATLAB® `curl` e `divergence` non evidenziano alcuna struttura.

4.3. Conclusioni

Il campo di moto medio e, in particolare, le strutture toroidali dei vortici sono coerenti rispetto a quanto atteso in condizioni di VRS e al-

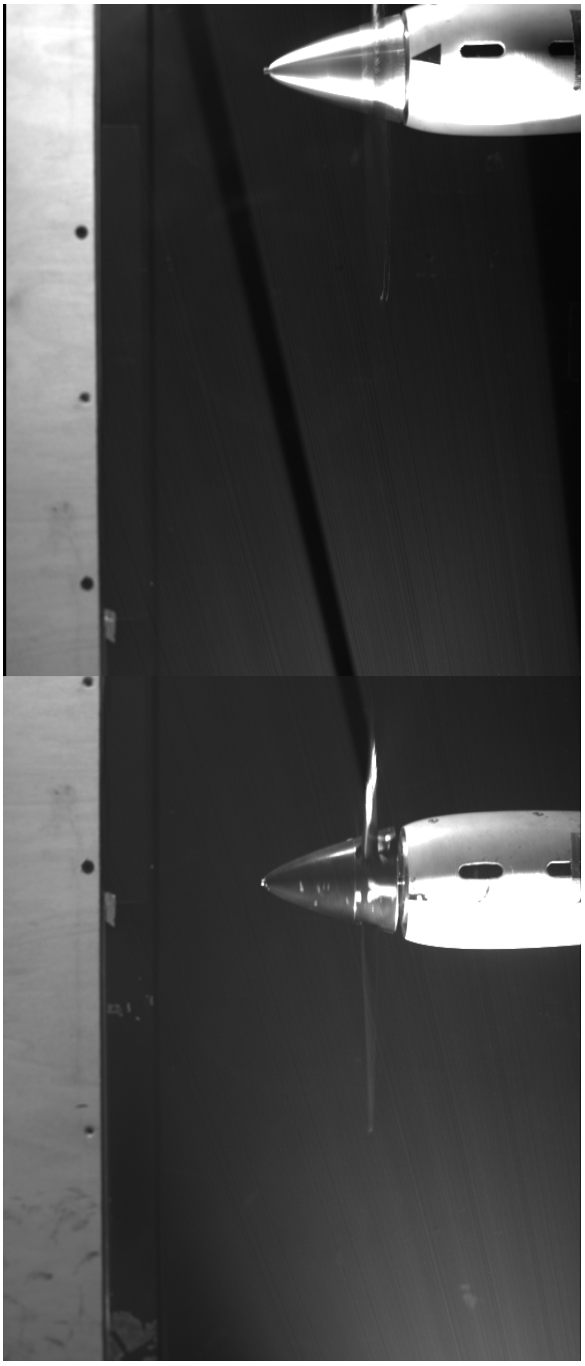


Figura 13: *Background* medio ricavato a partire da tutte e 100 le coppie di immagini acquisite.

l'elaborazione delle immagini eseguita mediante PIVlab. Si ritiene dunque che l'implementazione dello *script* MATLAB® funzioni correttamente, al netto delle difficoltà introdotte dalle disomogeneità nell'illuminazione e dai conseguenti errori sistematici, individuabili alla semplice ispezione dei risultati. Inoltre, in Sezione 3.2.2 si riporta un confronto tra il tempo di esecuzione della cross-correlazione calcolata nello spazio reale e quella svolta con l'ausilio della FFT: a parità

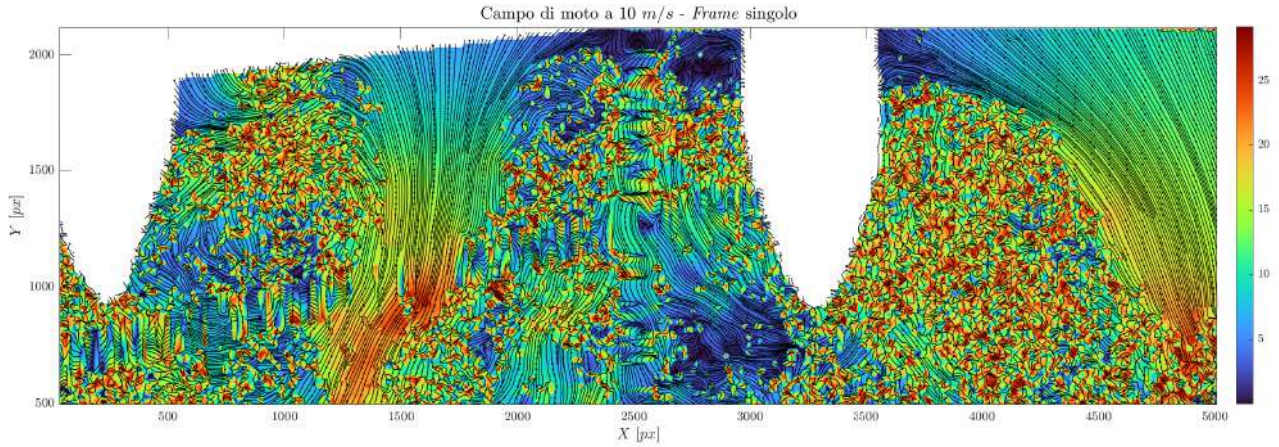
di risultati, se non in zone dove l'operazione risulta difficoltosa, la seconda strategia comporta un notevole risparmio computazionale e di tempo. In conclusione, il codice sviluppato risulta sufficiente ad ottenere alcune grandezze derivate dalle componenti di velocità, con riferimento alle Figure 10, 11b e 11c.

Riferimenti bibliografici

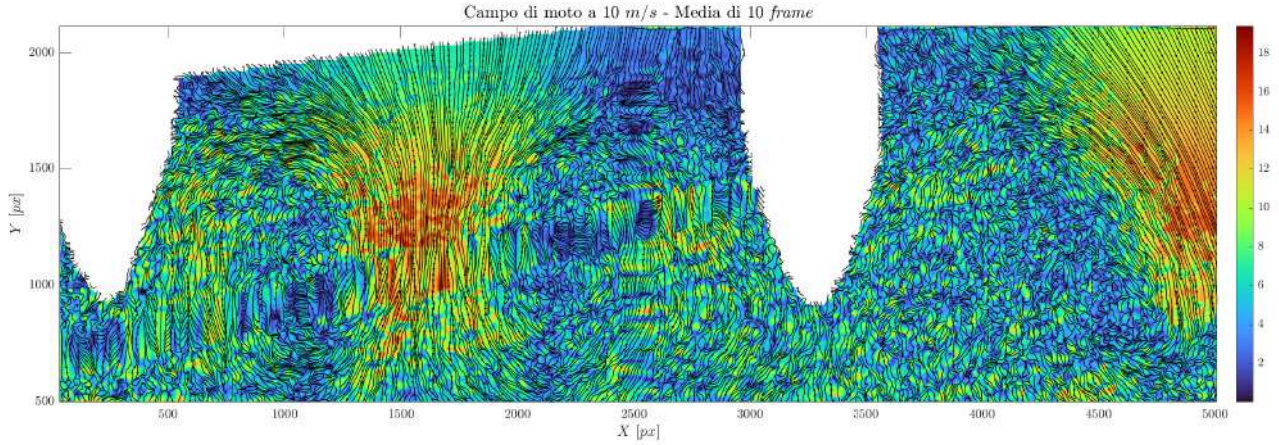
[1] ILA_5150. Specifiche della fotocamera ILA.PIV.sCMOS. `https://ila5150.de/uploads/media/5a573a13ae05a/ila-5150-data-scmos-clhs.pdf`. URL consultato il 13-01-2025.

[2] Cameron Tropea, Alexander L. Yarin, and John F. Foss, editors. *Springer Handbook of Experimental Fluid Dynamics*. Springer, Berlin, Heidelberg, 2007.

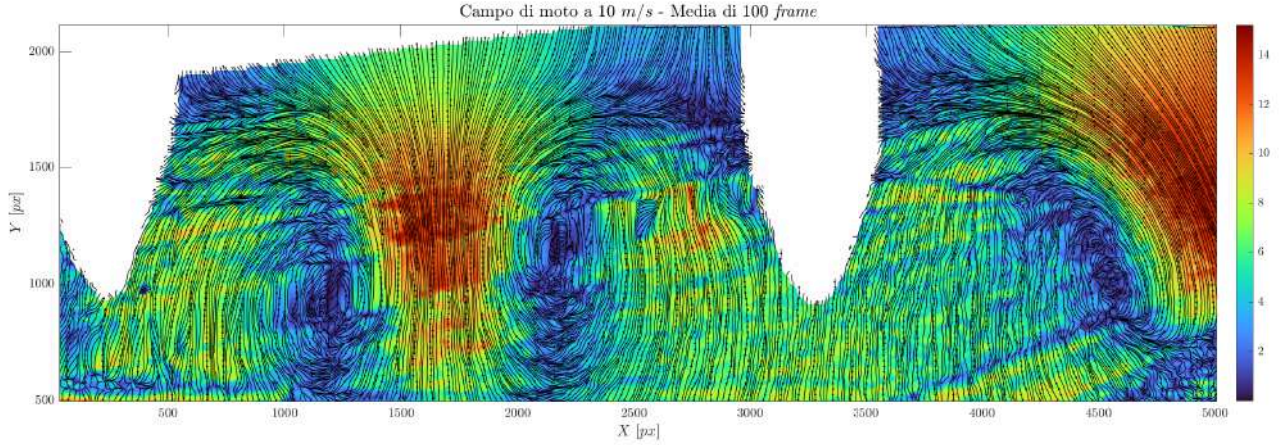
Appendice A: confronto dei risultati ottenuti mediando un numero variabile di immagini



(a)



(b)



(c)

Figura 14: Risultati ottenuti mediante la trasformata di Fourier per la prova sperimentale a 10 m/s. Correlazione di un singolo *frame* (a), media di 10 (b) e 100 (c) *frame*.

Appendice B: *script* MATLAB® per l'elaborazione dei dati sperimentali

Si include nel seguito il codice utilizzato per ottenere i risultati riportati nella presente relazione di laboratorio.

Codice per l'analisi PIV classica

```
close all
clear all
clc

%% inizializzazione delle cartelle contenenti le immagini e delle
   maschere
addpath 10ms\
load mascheratura_fascione.mat % maschera del supporto
load mascheratura_nacelle1.mat % maschera della nacelle destra
load mascheratura_nacelle2.mat % maschera della nacelle sinistra
load mascheratura_ombra1.mat % maschera ombra nelle immagini A
load mascheratura_ombra2.mat % maschera ombra nelle immagini B

maschera_ombra = min(maschera_ombra1+maschera_ombra2, 1);

%% importazione delle immagini 1
k = 1;
Cam1A = sprintf('Cam1_000%dA.b16', k);
Cam1A = readB16('10ms/Cam1', Cam1A);
Cam1B = sprintf('Cam1_000%dB.b16', k);
Cam1B = readB16('10ms/Cam1', Cam1B);
Cam2A = sprintf('Cam2_000%dA.b16', k);
Cam2A = readB16('10ms/Cam2', Cam2A);
Cam2B = sprintf('Cam2_000%dB.b16', k);
Cam2B = readB16('10ms/Cam2', Cam2B);

%% merging ed allineamento delle immagini delle due telecamere
npixely = 12;
npixelx = 45;

Cam2A(:, end-npixelx+1:end) = [];
Cam2A(end-npixely+1:end, :) = [];
Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

Cam2B(:, end-npixelx+1:end) = [];
Cam2B(end-npixely+1:end, :) = [];
Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');

image1 = [Cam2A, Cam1A];
image1 = imflatfield(image1, 2);

image2 = [Cam2B, Cam1B];
image2 = imflatfield(image2, 2);
```

```
%% padarray dell'immagine 2
image1(:, 1:3) = [];
image2(:, 1:3) = [];
image2 = padarray(image2, [16, 16], 0, 'both');

%% definizione dei parametri per l'analisi PIV
windowSize1 = 32; % Dimensione della finestra di interrogazione della
   prima immagine
windowSize2 = 64; % Dimensione della finestra di interrogazione della
   seconda immagine
overlap = windowSize1/2; % Overlap tra finestre (adjustable)

% dimensioni dell'immagine
[height, width] = size(image1);

% Calcono del numero di finestre
numWindowsX = floor((width - windowSize1) / (windowSize1 - overlap)) +
   1;
numWindowsY = floor((height - windowSize1) / (windowSize1 - overlap)) +
    1;

num_coppie = 100;

% inziazlizzazione dei vettori di spostamento
X = zeros(numWindowsY, numWindowsX, num_coppie);
Y = zeros(numWindowsY, numWindowsX, num_coppie);
Modulo = zeros(numWindowsY, numWindowsX, num_coppie);

%% ANALISI PIV IN PARALLELO
parfor ind = 1 : num_coppie
    if ind < 10
        Cam1A = sprintf('Cam1_000%dA.b16', ind);
        Cam1A = readB16('10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_000%dB.b16', ind);
        Cam1B = readB16('10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_000%dA.b16', ind);
        Cam2A = readB16('10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_000%dB.b16', ind);
        Cam2B = readB16('10ms/Cam2', Cam2B);
    elseif ind == 100
        Cam1A = sprintf('Cam1_0%dA.b16', ind);
        Cam1A = readB16('10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_0%dB.b16', ind);
        Cam1B = readB16('10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_0%dA.b16', ind);
        Cam2A = readB16('10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_0%dB.b16', ind);
        Cam2B = readB16('10ms/Cam2', Cam2B);
    else
        Cam1A = sprintf('Cam1_00%dA.b16', ind);
```

```
    Cam1A = readB16('10ms/Cam1', Cam1A);
    Cam1B = sprintf('Cam1_00%dB.b16', ind);
    Cam1B = readB16('10ms/Cam1', Cam1B);
    Cam2A = sprintf('Cam2_00%dA.b16', ind);
    Cam2A = readB16('10ms/Cam2', Cam2A);
    Cam2B = sprintf('Cam2_00%dB.b16', ind);
    Cam2B = readB16('10ms/Cam2', Cam2B);
end

%% merging ed allineamento delle immagini delle due telecamere
npixely = 12;
npixelx = 45;

Cam2A(:, end-npixelx+1:end) = [];
Cam2A(end-npixely+1:end, :) = [];
Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

Cam2B(:, end-npixelx+1:end) = [];
Cam2B(end-npixely+1:end, :) = [];
Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');

a = mean(Cam2A);
b = mean(Cam1A);

Cam2A = max(0, Cam2A - (a(end) - b(1)));
Cam2B = max(0, Cam2B - (a(end) - b(1)));

image1 = [Cam2A, Cam1A];
image1 = imflatfield(image1, 2);

image2 = [Cam2B, Cam1B];
image2 = imflatfield(image2, 2);

%% padarray dell'immagine 2
image1(:, 1:3) = [];
image2(:, 1:3) = [];
image2 = padarray(image2, [(windowSize2 - windowSize1)/2, (
   windowSize2 - windowSize1)/2], 0, 'both');

%%
for i = 1:numWindowsY
    for j = 1:numWindowsX
        % Definizione delle coordinate delle sottofinestre per la
           prima
        % immagine
        xStart1 = (j - 1) * (windowSize1 - overlap) + 1;
        yStart1 = (i - 1) * (windowSize1 - overlap) + 1;
        xEnd1 = xStart1 + windowSize1 - 1;
        yEnd1 = yStart1 + windowSize1 - 1;

        % Definizione delle coordinate delle sottofinestre per la
```

```
% seconda immagine
xStart2 = xStart1;
yStart2 = yStart1;
xEnd2 = xStart2 + windowSize2 - 1;
yEnd2 = yStart2 + windowSize2 - 1;

% Estrazione delle sottofinestre
roi1 = double(image1(yStart1:yEnd1, xStart1:xEnd1));
roi2 = double(image2(yStart2:yEnd2, xStart2:xEnd2));
media_roi1 = mean(mean(roi1));
media_roi2 = mean(mean(roi2));

crossCorr = zeros(windowSize1, windowSize1);
% Cross-correlazione
for k = 1 : windowSize1+1
    for l = 1 : windowSize1+1
        xStart3 = l;
        yStart3 = k;
        xEnd3 = l + windowSize1 - 1;
        yEnd3 = k + windowSize1 - 1;
        subroi2 = double(roi2(yStart3:yEnd3, xStart3:xEnd3)
           );
        matrix_mult = max(0, (roi1-media_image1)) .* max(0,
            (subroi2-media_image2));
        corr = sum(sum(matrix_mult));
        crossCorr(k, l) = corr;
    end
end

% Identificazione del picco di cross-correlation
[maxVal, maxIdx] = max(crossCorr, [], 'all');
[peakY, peakX] = ind2sub(size(crossCorr), maxIdx);

% subpixel interpolation
if peakX == 1 || peakX == (windowSize1 + 1)
    peakX_new = peakX;
else
    x1 = peakX-1;
    x2 = peakX;
    x3 = peakX+1;
    y1 = crossCorr(peakY, x1); % value at peak-1 location
    y2 = crossCorr(peakY, x2); % value at peak location
    y3 = crossCorr(peakY, x3); % value at peak+1 location
    [peakX_new, ~, ~] = three_point_gaussian_fit(x1, y1, x2
       , y2, x3, y3);
    if ~isreal(peakX_new) || isnan(peakX_new)
        peakX_new = peakX;
    end
end

if peakY == 1 || peakY == (windowSize1 + 1)
```

```
                peakY_new = peakY;
            else
                x1 = peakY-1;
                x2 = peakY;
                x3 = peakY+1;
                y1 = crossCorr(x1, peakX); % value at peak-1 location
                y2 = crossCorr(x2, peakX); % value at peak location
                y3 = crossCorr(x3, peakX); % value at peak+1 location
                [peakY_new, ~, ~] = three_point_gaussian_fit(x1, y1, x2
                    , y2, x3, y3);
                if ~isreal(peakY_new) || isnan(peakY_new)
                    peakY_new = peakY;
                end
            end

            peakX = peakX_new;
            peakY = peakY_new;

            % Calcolo degli spostamenti
            displacementX = peakX - 1 - (windowSize2 - windowSize1)/2;
            displacementY = -(peakY - 1 - (windowSize2 - windowSize1)
                /2);

            if maxVal == 0
                displacementX = 0;
                displacementY = 0;
            end

            % identificazione delle regioni mascherate e imposizione di
            % spostamento nullo
            if maschera_fascione((i*overlap), (j*overlap)) == 0 ||
                maschera_nacelle1((i*overlap), (j*overlap)) == 0 ||
                maschera_nacelle2((i*overlap), (j*overlap)) == 0
                displacementX = 0;
                displacementY = 0;
            end

            % Risultati
            X(i, j, ind) = displacementX;
            Y(i, j, ind) = displacementY;
            Modulo(i, j, ind) = sqrt(displacementX^2+displacementY^2);
        end
    end
end

%% Interpolazione regione di congiunzione
for ind = 1 : num_coppie
    for i = 1 : numWindowsY
        gap = 6; % numero di sottofinestre eliminate + 1
        val_pre_X = X(i, round(numWindowsX/2)-(gap/2), ind);
        val_post_X = X(i, round(numWindowsX/2)+(gap/2), ind);
```

```
        val_pre_Y = Y(i, round(numWindowsX/2)-(gap/2), ind);
        val_post_Y = Y(i, round(numWindowsX/2)+(gap/2), ind);
        for j = 1 : gap-1
            % interpolazione lineare
            X(i, round(numWindowsX/2)-(gap/2)+j, ind) = val_pre_X + (
               val_post_X - val_pre_X) * (j * overlap) / (gap * overlap
               );
            Y(i, round(numWindowsX/2)-(gap/2)+j, ind) = val_pre_Y + (
               val_post_Y - val_pre_Y) * (j * overlap) / (gap * overlap
               );
        end
    end
end

%% interpolazione zona ombra
check = min(maschera_ombra);
for ind = 1 : num_coppie
    for j = 1 : numWindowsX
        if check(j*overlap) == 0
            indices = unique(round(find(any(~maschera_ombra(:,j*overlap
               ),2))./overlap));
            val_pre_X = X(indices(1)-1, j, ind);
            val_post_X = X(indices(end)+1, j, ind);
            val_pre_Y = Y(indices(1)-1, j, ind);
            val_post_Y = Y(indices(end)+1, j, ind);
            differenza = (indices(end)+1) - (indices(1)-1);
            for i = 1 : length(indices)
                % interpolazione lineare
                X(indices(i), j, ind) = val_pre_X + (val_post_X -
                   val_pre_X) * (i * overlap) / (differenza * overlap);
                Y(indices(i), j, ind) = val_pre_Y + (val_post_Y -
                   val_pre_Y) * (i * overlap) / (differenza * overlap);
            end
        end
    end
end

%% Outliers
modulo_max = 20;
threshold = 2;
toll = 0.1;
% Precalcolo di tutte le posizione relative degli 8 punti intorno
neighbors = [-1 -1; -1 0; -1 1; 0 -1; 0 1; 1 -1; 1 0; 1 1];

for ind = 1 : num_coppie
    for i = 2:numWindowsY-1
        for j = 2:numWindowsX-1
            % Estrazione dei valori nell'intorno
            val_vect = Modulo(i-1:i+1, j-1:j+1, ind);
            val_vect = val_vect([1, 2, 3, 4, 6, 7, 8, 9]);
```

```
            % median teest
            residual = abs(Modulo(i, j, ind) - median(val_vect)) / (
               median(abs(Modulo(i, j, ind) - val_vect)) + toll);

            if residual > threshold || Modulo(i, j, ind) >= modulo_max
                % Calcolo delle distanze
                d = sqrt([2*overlap^2, overlap^2, 2*overlap^2, overlap
                   ^2, overlap^2, 2*overlap^2, overlap^2, 2*overlap^2])
                   ;
                somma_d = sum(1 ./ d); % somma degli inversi delle
                   distanze

                % Weights
                w = (1 ./ d) / somma_d;

                % Weighted average for X and Y
                weight_sum = sum(w);
                X_vect = X(i-1:i+1, j-1:j+1, ind);
                X_vect = X_vect([1, 2, 3, 4, 6, 7, 8, 9]);
                X(i, j, ind) = sum(w .* X_vect) / weight_sum;
                Y_vect = Y(i-1:i+1, j-1:j+1, ind);
                Y_vect = Y_vect([1, 2, 3, 4, 6, 7, 8, 9]);
                Y(i, j, ind) = sum(w .* Y_vect) / weight_sum;
            end
        end
    end
end

%% Conversione da px a mm e generazione del campo di velocit
dist_pixel = 119;
dist_mm = 17.57;
M = dist_pixel/dist_mm;
X = X/M*1e-3;
Y = Y/M*1e-3;
U = X / (100*1e-6);
V = Y / (100*1e-6);

%% Calcolo del moto medio e delle fluttuazioni
U_medio = mean(U, 3);
V_medio = mean(V, 3);
u = U - U_medio;
v = V - V_medio;

%% Calcolo delle energie cinetiche
E = 0.5 * (U_medio.^2 + V_medio.^2); % mean-flow kinetic energy
u_prime_squared = u.^2;
v_prime_squared = v.^2;
u_var = mean(u_prime_squared, 3);
v_var = mean(v_prime_squared, 3);
TKE = 0.5 * sqrt(u_var+v_var); % turbulent kinetic energy
```

Codice per l'analisi PIV mediante FFT

```
close all
clear all
clc

%% inizializzazione delle cartelle contenenti le immagini e delle
   maschere
addpath 10ms\
load mascheratura_fascione.mat % maschera del supporto
load mascheratura_nacelle1.mat % maschera della nacelle destra
load mascheratura_nacelle2.mat % maschera della nacelle sinistra
load mascheratura_ombra1.mat % maschera ombra nelle immagini A
load mascheratura_ombra2.mat % maschera ombra nelle immagini B

maschera_ombra = min(maschera_ombra1+maschera_ombra2, 1);

%% importazione delle immagini 1
k = 1;
Cam1A = sprintf('Cam1_000%dA.b16', k);
Cam1A = readB16('10ms/Cam1', Cam1A);
Cam1B = sprintf('Cam1_000%dB.b16', k);
Cam1B = readB16('10ms/Cam1', Cam1B);
Cam2A = sprintf('Cam2_000%dA.b16', k);
Cam2A = readB16('10ms/Cam2', Cam2A);
Cam2B = sprintf('Cam2_000%dB.b16', k);
Cam2B = readB16('10ms/Cam2', Cam2B);

%% merging ed allineamento delle immagini delle due telecamere
npixely = 12;
npixelx = 45;

Cam2A(:, end-npixelx+1:end) = [];
Cam2A(end-npixely+1:end, :) = [];
Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

Cam2B(:, end-npixelx+1:end) = [];
Cam2B(end-npixely+1:end, :) = [];
Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');

image1 = [Cam2A, Cam1A];
image1 = imflatfield(image1, 2);

image2 = [Cam2B, Cam1B];
image2 = imflatfield(image2, 2);

%% padarray dell'immagine 2
image1(:, 1:3) = [];
image2(:, 1:3) = [];
image2 = padarray(image2, [16, 16], 0, 'both');

%% definizione dei parametri per l'analisi PIV
```

```
windowSize1 = 32; % Dimensione della finestra di interrogazione della
   prima immagine
windowSize2 = 64; % Dimensione della finestra di interrogazione della
   seconda immagine
overlap = windowSize1/2; % Overlap tra finestre (adjustable)

% dimensioni dell'immagine
[height, width] = size(image1);

% Calcono del numero di finestre
numWindowsX = floor((width - windowSize1) / (windowSize1 - overlap)) +
   1;
numWindowsY = floor((height - windowSize1) / (windowSize1 - overlap)) +
    1;

num_coppie = 100;

% inziazlizzazione dei vettori di spostamento
X = zeros(numWindowsY, numWindowsX, num_coppie);
Y = zeros(numWindowsY, numWindowsX, num_coppie);
Modulo = zeros(numWindowsY, numWindowsX, num_coppie);

%% ANALISI PIV IN PARALLELO
parfor ind = 1 : num_coppie
    if ind < 10
        Cam1A = sprintf('Cam1_000%dA.b16', ind);
        Cam1A = readB16('10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_000%dB.b16', ind);
        Cam1B = readB16('10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_000%dA.b16', ind);
        Cam2A = readB16('10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_000%dB.b16', ind);
        Cam2B = readB16('10ms/Cam2', Cam2B);
    elseif ind == 100
        Cam1A = sprintf('Cam1_0%dA.b16', ind);
        Cam1A = readB16('10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_0%dB.b16', ind);
        Cam1B = readB16('10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_0%dA.b16', ind);
        Cam2A = readB16('10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_0%dB.b16', ind);
        Cam2B = readB16('10ms/Cam2', Cam2B);
    else
        Cam1A = sprintf('Cam1_00%dA.b16', ind);
        Cam1A = readB16('10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_00%dB.b16', ind);
        Cam1B = readB16('10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_00%dA.b16', ind);
        Cam2A = readB16('10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_00%dB.b16', ind);
        Cam2B = readB16('10ms/Cam2', Cam2B);
```

```
end

%% merging ed allineamento delle immagini delle due telecamere
npixely = 12;
npixelx = 45;

Cam2A(:, end-npixelx+1:end) = [];
Cam2A(end-npixely+1:end, :) = [];
Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

Cam2B(:, end-npixelx+1:end) = [];
Cam2B(end-npixely+1:end, :) = [];
Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');

a = mean(Cam2A);
b = mean(Cam1A);

Cam2A = max(0, Cam2A - (a(end) - b(1)));
Cam2B = max(0, Cam2B - (a(end) - b(1)));

image1 = [Cam2A, Cam1A];
image1 = imflatfield(image1, 2);

image2 = [Cam2B, Cam1B];
image2 = imflatfield(image2, 2);

%% padarray dell'immagine 2
image1(:, 1:3) = [];
image2(:, 1:3) = [];
image2 = padarray(image2, [(windowSize2 - windowSize1)/2, (
   windowSize2 - windowSize1)/2], 0, 'both');

%%
for i = 1:numWindowsY
    for j = 1:numWindowsX
        % Definizione delle coordinate delle sottofinestre per la
           prima
        % immagine
        xStart1 = (j - 1) * (windowSize1 - overlap) + 1;
        yStart1 = (i - 1) * (windowSize1 - overlap) + 1;
        xEnd1 = xStart1 + windowSize1 - 1;
        yEnd1 = yStart1 + windowSize1 - 1;

        % Definizione delle coordinate delle sottofinestre per la
        % seconda immagine
        xStart2 = xStart1;
        yStart2 = yStart1;
        xEnd2 = xStart2 + windowSize2 - 1;
        yEnd2 = yStart2 + windowSize2 - 1;

        % Estrazione delle sottofinestre
```

```
roi1 = double(image1(yStart1:yEnd1, xStart1:xEnd1));
roi2 = double(image2(yStart2:yEnd2, xStart2:xEnd2));
media_roi1 = mean(mean(roi1));
media_roi2 = mean(mean(roi2));
roi1 = padarray(roi1, [(windowSize2 - windowSize1)/2, (
   windowSize2 - windowSize1)/2], 0, 'both');

% FFT e costruzione del piano di correlazione
crossCorr = real(ifft2(conj(fft2(max(0, roi1-media_roi1)))
   .*fft2(max(0, roi2-media_roi2))));
crossCorr = fftshift(fftshift(crossCorr, 1), 2);
crossCorr = crossCorr(1+(windowSize2-windowSize1)/2 : 1+(
   windowSize2-windowSize1)/2+windowSize1, 1+(windowSize2-
   windowSize1)/2 : 1+(windowSize2-windowSize1)/2+
   windowSize1);


% Identificazione del picco di cross-correlation
[maxVal, maxIdx] = max(crossCorr, [], 'all');
[peakY, peakX] = ind2sub(size(crossCorr), maxIdx);

% subpixel interpolation
if peakX == 1 || peakX == (windowSize1 + 1)
    peakX_new = peakX;
else
    x1 = peakX-1;
    x2 = peakX;
    x3 = peakX+1;
    y1 = crossCorr(peakY, x1); % value at peak-1 location
    y2 = crossCorr(peakY, x2); % value at peak location
    y3 = crossCorr(peakY, x3); % value at peak+1 location
    [peakX_new, ~, ~] = three_point_gaussian_fit(x1, y1, x2
       , y2, x3, y3);
    if ~isreal(peakX_new) || isnan(peakX_new)
        peakX_new = peakX;
    end
end

if peakY == 1 || peakY == (windowSize1 + 1)
    peakY_new = peakY;
else
    x1 = peakY-1;
    x2 = peakY;
    x3 = peakY+1;
    y1 = crossCorr(x1, peakX); % value at peak-1 location
    y2 = crossCorr(x2, peakX); % value at peak location
    y3 = crossCorr(x3, peakX); % value at peak+1 location
    [peakY_new, ~, ~] = three_point_gaussian_fit(x1, y1, x2
       , y2, x3, y3);
    if ~isreal(peakY_new) || isnan(peakY_new)
        peakY_new = peakY;
```

```
                end
            end

            peakX = peakX_new;
            peakY = peakY_new;

            % Calcolo degli spostamenti
            displacementX = peakX - 1 - (windowSize2 - windowSize1)/2;
            displacementY = -(peakY - 1 - (windowSize2 - windowSize1)
               /2);

            if maxVal == 0
                displacementX = 0;
                displacementY = 0;
            end

            % identificazione delle regioni mascherate e imposizione di
            % spostamento nullo
            if maschera_fascione((i*overlap), (j*overlap)) == 0 ||
               maschera_nacelle1((i*overlap), (j*overlap)) == 0 ||
               maschera_nacelle2((i*overlap), (j*overlap)) == 0
                displacementX = 0;
                displacementY = 0;
            end

            % Risultati
            X(i, j, ind) = displacementX;
            Y(i, j, ind) = displacementY;
            Modulo(i, j, ind) = sqrt(displacementX^2+displacementY^2);
        end
    end
end

%% Interpolazione regione di congiunzione
for ind = 1 : num_coppie
    for i = 1 : numWindowsY
        gap = 6; % numero di sottofinestre eliminate + 1
        val_pre_X = X(i, round(numWindowsX/2)-(gap/2), ind);
        val_post_X = X(i, round(numWindowsX/2)+(gap/2), ind);
        val_pre_Y = Y(i, round(numWindowsX/2)-(gap/2), ind);
        val_post_Y = Y(i, round(numWindowsX/2)+(gap/2), ind);
        for j = 1 : gap-1
            % interpolazione lineare
            X(i, round(numWindowsX/2)-(gap/2)+j, ind) = val_pre_X + (
               val_post_X - val_pre_X) * (j * overlap) / (gap * overlap
               );
            Y(i, round(numWindowsX/2)-(gap/2)+j, ind) = val_pre_Y + (
               val_post_Y - val_pre_Y) * (j * overlap) / (gap * overlap
               );
        end
    end
```

```
end

%% interpolazione zona ombra
check = min(maschera_ombra);
for ind = 1 : num_coppie
    for j = 1 : numWindowsX
        if check(j*overlap) == 0
            indices = unique(round(find(any(~maschera_ombra(:,j*overlap
               ),2))./overlap));
            val_pre_X = X(indices(1)-1, j, ind);
            val_post_X = X(indices(end)+1, j, ind);
            val_pre_Y = Y(indices(1)-1, j, ind);
            val_post_Y = Y(indices(end)+1, j, ind);
            differenza = (indices(end)+1) - (indices(1)-1);
            for i = 1 : length(indices)
                % interpolazione lineare
                X(indices(i), j, ind) = val_pre_X + (val_post_X -
                   val_pre_X) * (i * overlap) / (differenza * overlap);
                Y(indices(i), j, ind) = val_pre_Y + (val_post_Y -
                   val_pre_Y) * (i * overlap) / (differenza * overlap);
            end
        end
    end
end

%% Outliers
modulo_max = 20;
threshold = 2;
toll = 0.1;
% Precalcolo di tutte le posizione relative degli 8 punti intorno
neighbors = [-1 -1; -1 0; -1 1; 0 -1; 0 1; 1 -1; 1 0; 1 1];

for ind = 1 : num_coppie
    for i = 2:numWindowsY-1
        for j = 2:numWindowsX-1
            % Estrazione dei valori nell'intorno
            val_vect = Modulo(i-1:i+1, j-1:j+1, ind);
            val_vect = val_vect([1, 2, 3, 4, 6, 7, 8, 9]);

            % median teest
            residual = abs(Modulo(i, j, ind) - median(val_vect)) / (
               median(abs(Modulo(i, j, ind) - val_vect)) + toll);

            if residual > threshold || Modulo(i, j, ind) >= modulo_max
                % Calcolo delle distanze
                d = sqrt([2*overlap^2, overlap^2, 2*overlap^2, overlap
                   ^2, overlap^2, 2*overlap^2, overlap^2, 2*overlap^2])
                   ;
                somma_d = sum(1 ./ d); % somma degli inversi delle
                   distanze
```

```
                % Weights
                w = (1 ./ d) / somma_d;

                % Weighted average for X and Y
                weight_sum = sum(w);
                X_vect = X(i-1:i+1, j-1:j+1, ind);
                X_vect = X_vect([1, 2, 3, 4, 6, 7, 8, 9]);
                X(i, j, ind) = sum(w .* X_vect) / weight_sum;
                Y_vect = Y(i-1:i+1, j-1:j+1, ind);
                Y_vect = Y_vect([1, 2, 3, 4, 6, 7, 8, 9]);
                Y(i, j, ind) = sum(w .* Y_vect) / weight_sum;
            end
        end
    end
end

%% Conversione da px a mm e generazione del campo di velocita'
dist_pixel = 119;
dist_mm = 17.57;
M = dist_pixel/dist_mm;
X = X/M*1e-3;
Y = Y/M*1e-3;
U = X / (100*1e-6);
V = Y / (100*1e-6);

%% Calcolo del moto medio e delle fluttuazioni
U_medio = mean(U, 3);
V_medio = mean(V, 3);
u = U - U_medio;
v = V - V_medio;

%% Calcolo delle energie cinetiche
E = 0.5 * (U_medio.^2 + V_medio.^2); % mean-flow kinetic energy
u_prime_squared = u.^2;
v_prime_squared = v.^2;
u_var = mean(u_prime_squared, 3);
v_var = mean(v_prime_squared, 3);
TKE = 0.5 * sqrt(u_var+v_var); % turbulent kinetic energy
```

Funzione per l'interpolazione *subpixel*

```
function [mu, A, B] = three_point_gaussian_fit(x1, y1, x2, y2, x3, y3)
%
% This function finds the coefficients mu, A, B such that the 3 input
% points belong to the gaussian function y = A * exp(-(x-mu)^2 / B)
%
% Input:
% - x1       1st point, x
% - y1       1st point, y
% - x2       2nd point, x
% - y2       2nd point, y
% - x3       3rd point, x
% - y3       3rd point, y
%
% Output:
% - mu       mean value of the gaussian function => maximum point
% - A
% - B

mu = ( (x3^2-x2^2)*log(y1/y2)+(x1^2-x2^2)*log(y2/y3) ) / (2*( (x1-x2)*
   log(y2/y3)+(x3-x2)*log(y1/y2) ));
B = (x2^2 - x1^2 + 2*(x1-x2)*mu) / log(y1/y2);
A = y1 * exp((x1-mu)^2/B);
```

Modifica del codice per l'analisi PIV mediante FFT per l'interrogazione *Multi-pass*

```
%% Multipass

image3 = padarray(image2, [(windowSize2 -windowSize1)/2, (windowSize2 -
   windowSize1)/2], 0, 'both');

% Definizione delle coordinate delle sottofinestre per il multipass
xStart3 = xStart2 + (windowSize2 - windowSize1)/2 + round(displacementX
   );
yStart3 = yStart2 + (windowSize2 - windowSize1)/2 + round(displacementY
   );
xEnd3 = xEnd2 + (windowSize2 - windowSize1)/2 + round(displacementX);
yEnd3 = yEnd2 + (windowSize2 - windowSize1)/2 + round(displacementY);

roi3 = double(image3(yStart3:yEnd3, xStart3:xEnd3));
media_roi3 = mean(mean(roi3));

% FFT e costruzione del piano di correlazione
crossCorr = real(ifft2(conj(fft2(max(0, roi1-media_roi1))).*fft2(max(0,
    roi3-media_roi3))));
crossCorr = fftshift(fftshift(crossCorr, 1), 2);
crossCorr = crossCorr(1+(windowSize2-windowSize1)/2 : 1+(windowSize2-
   windowSize1)/2+windowSize1, 1+(windowSize2-windowSize1)/2 : 1+(
   windowSize2-windowSize1)/2+windowSize1);


% Ricerca del picco di correlazione
[maxVal, maxIdx] = max(crossCorr, [], 'all');
[peakY, peakX] = ind2sub(size(crossCorr), maxIdx);

% subpixel interpolation
if peakX == 1 || peakX == (windowSize1 + 1)
    peakX_new = peakX;
else
    x1 = peakX-1;
    x2 = peakX;
    x3 = peakX+1;
    y1 = crossCorr(peakY, x1); % value at peak-1 location
    y2 = crossCorr(peakY, x2); % value at peak location
    y3 = crossCorr(peakY, x3); % value at peak+1 location
    [peakX_new, ~, ~] = three_point_gaussian_fit(x1, y1, x2, y2, x3, y3
       );
    if ~isreal(peakX_new) || isnan(peakX_new)
        peakX_new = peakX;
    end
end

if peakY == 1 || peakY == (windowSize1 + 1)
    peakY_new = peakY;
else
    x1 = peakY-1;
```

```
    x2 = peakY;
    x3 = peakY+1;
    y1 = crossCorr(x1, peakX); % value at peak-1 location
    y2 = crossCorr(x2, peakX); % value at peak location
    y3 = crossCorr(x3, peakX); % value at peak+1 location
    [peakY_new, ~, ~] = three_point_gaussian_fit(x1, y1, x2, y2, x3, y3
       );
    if ~isreal(peakY_new) || isnan(peakY_new)
        peakY_new = peakY;
    end
end

peakX = peakX_new;
peakY = peakY_new;

% Calcolo degli spostamenti del multipass
displacementX_multi = peakX - 1 - (windowSize2 - windowSize1)/2;
displacementY_multi = -(peakY - 1 - (windowSize2 - windowSize1)/2);

if maxVal == 0
    displacementX_multi = 0;
    displacementY_multi = 0;
end

% identificazione delle regioni mascherate e imposizione di spostamento
    nullo
if maschera_fascione((i*overlap), (j*overlap)) == 0 ||
   maschera_nacelle1((i*overlap), (j*overlap)) == 0 ||
   maschera_nacelle2((i*overlap), (j*overlap)) == 0
    displacementX_multi = 0;
    displacementY_multi = 0;
end

%% Store results
X(i, j, ind) = displacementX + displacementX_multi;
Y(i, j, ind) = displacementY + displacementY_multi;
Modulo(i, j, ind) = sqrt(X(i, j, ind)^2+Y(i, j, ind)^2);
```

Codice per salvare le immagini in formato .b16 leggibile da PIVlab

```
% Script MATLAB realizzato per il Laboratorio 0 ( Misure PIV ) del
   corso di Fluidodinamica Sperimentale , A . A . 2024-25.
% Autori : Federico Poleni , Alberto Vacca , Giacomo Visconti ,
% Marco Zappoli


clear
close all
clc


%% Carica le immagini acquisite, unisce le foto delle due camere e
   salva in formato leggibile da PIVlab

% Numero pixel necessario per unire le immagini
npixely = 12;
npixelx = 45;

for ind = 1 : 100
    if ind < 10
        Cam1A = sprintf('Cam1_000%dA.b16', ind);
        Cam1A = readB16('Foto/10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_000%dB.b16', ind);
        Cam1B = readB16('Foto/10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_000%dA.b16', ind);
        Cam2A = readB16('Foto/10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_000%dB.b16', ind);
        Cam2B = readB16('Foto/10ms/Cam2', Cam2B);

        Cam2A(:, end-npixelx+1:end) = [];
        Cam2A(end-npixely+1:end, :) = [];
        Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

        Cam2B(:, end-npixelx+1:end) = [];
        Cam2B(end-npixely+1:end, :) = [];
        Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');

        image1 = [Cam2A, Cam1A]';
        image2 = [Cam2B, Cam1B]';

        % Caratteristiche header, dimensione file e percorso
        HeadLength = 32;
        file_size_MB1 = numel(image1) * 2 + HeadLength; % Ogni pixel
           occupa 2 byte (16 bit)
        rowimm1 = size(image1,1);
        colimm1 = size(image1,2);

        file_size_MB2 = numel(image2) * 2 + HeadLength; % Ogni pixel
           occupa 2 byte (16 bit)
        rowimm2 = size(image2,1);
```

```
        colimm2 = size(image2,2);

        folderPath = 'unite';
        CamJA = sprintf('CamJ_000%dA.b16', ind);
        CamJB = sprintf('CamJ_000%dB.b16', ind);
        % Definisce il percorso completo per il file.b16
        filePath1 = fullfile(folderPath, CamJA);
        filePath2 = fullfile(folderPath, CamJB);

        fileID1 = fopen(filePath1, 'wb');
        fileID2 = fopen(filePath2, 'wb');


        % Scrive le immagini in un file binario
        fwrite(fileID1,'PCO-', 'char');
        fwrite(fileID1,file_size_MB1, 'uint32');
        fwrite(fileID1,HeadLength, 'uint32');
        fwrite(fileID1,rowimm1, 'uint32');
        fwrite(fileID1,colimm1, 'uint32');
        fwrite(fileID1, 0, 'uint32');
        fwrite(fileID1,image1,"uint16");

        fwrite(fileID2,'PCO-', 'char');
        fwrite(fileID2,file_size_MB2, 'uint32');
        fwrite(fileID2,HeadLength, 'uint32');
        fwrite(fileID2,rowimm2, 'uint32');
        fwrite(fileID2,colimm2, 'uint32');
        fwrite(fileID2, 0, 'uint32');
        fwrite(fileID2,image2,"uint16");
        % Chiude i files
        fclose(fileID1);
        fclose(fileID2);

    elseif ind==100
        Cam1A = sprintf('Cam1_0%dA.b16', ind);
        Cam1A = readB16('Foto/10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_0%dB.b16', ind);
        Cam1B = readB16('Foto/10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_0%dA.b16', ind);
        Cam2A = readB16('Foto/10ms/Cam2', Cam2A);
        Cam2B = sprintf('Cam2_0%dB.b16', ind);
        Cam2B = readB16('Foto/10ms/Cam2', Cam2B);

        Cam2A(:, end-npixelx+1:end) = [];
        Cam2A(end-npixely+1:end, :) = [];
        Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

        Cam2B(:, end-npixelx+1:end) = [];
        Cam2B(end-npixely+1:end, :) = [];
        Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');
```

```
        image1 = [Cam2A, Cam1A]';
        image2 = [Cam2B, Cam1B]';

        % Caratteristiche header, dimensione file e percorso
        HeadLength = 32;
        file_size_MB1 = numel(image1) * 2 + HeadLength; % Ogni pixel
           occupa 2 byte (16 bit)
        rowimm1 = size(image1,1);
        colimm1 = size(image1,2);

        file_size_MB2 = numel(image2) * 2 + HeadLength; % Ogni pixel
           occupa 2 byte (16 bit)
        rowimm2 = size(image2,1);
        colimm2 = size(image2,2);

        folderPath = 'unite';
        CamJA = sprintf('CamJ_0%dA.b16', ind);
        CamJB = sprintf('CamJ_0%dB.b16', ind);
        % Definisce il percorso completo per il file.b16
        filePath1 = fullfile(folderPath, CamJA);
        filePath2 = fullfile(folderPath, CamJB);

        fileID1 = fopen(filePath1, 'wb');
        fileID2 = fopen(filePath2, 'wb');


        % Scrive le immagini in un file binario
        fwrite(fileID1,'PCO-', 'char');
        fwrite(fileID1,file_size_MB1, 'uint32');
        fwrite(fileID1,HeadLength, 'uint32');
        fwrite(fileID1,rowimm1, 'uint32');
        fwrite(fileID1,colimm1, 'uint32');
        fwrite(fileID1, 0, 'uint32');
        fwrite(fileID1,image1,"uint16");

        fwrite(fileID2,'PCO-', 'char');
        fwrite(fileID2,file_size_MB2, 'uint32');
        fwrite(fileID2,HeadLength, 'uint32');
        fwrite(fileID2,rowimm2, 'uint32');
        fwrite(fileID2,colimm2, 'uint32');
        fwrite(fileID2, 0, 'uint32');
        fwrite(fileID2,image2,"uint16");
        % Chiude i files
        fclose(fileID1);
        fclose(fileID2);
    else
        Cam1A = sprintf('Cam1_00%dA.b16', ind);
        Cam1A = readB16('Foto/10ms/Cam1', Cam1A);
        Cam1B = sprintf('Cam1_00%dB.b16', ind);
        Cam1B = readB16('Foto/10ms/Cam1', Cam1B);
        Cam2A = sprintf('Cam2_00%dA.b16', ind);
```

```
Cam2A = readB16('Foto/10ms/Cam2', Cam2A);
Cam2B = sprintf('Cam2_00%dB.b16', ind);
Cam2B = readB16('Foto/10ms/Cam2', Cam2B);

Cam2A(:, end-npixelx+1:end) = [];
Cam2A(end-npixely+1:end, :) = [];
Cam2A = padarray(Cam2A, [npixely, 0], 0, 'pre');

Cam2B(:, end-npixelx+1:end) = [];
Cam2B(end-npixely+1:end, :) = [];
Cam2B = padarray(Cam2B, [npixely, 0], 0, 'pre');

image1 = [Cam2A, Cam1A]';
image2 = [Cam2B, Cam1B]';

% Caratteristiche header, dimensione file e percorso
HeadLength = 32;
file_size_MB1 = numel(image1) * 2 + HeadLength; % Ogni pixel
   occupa 2 byte (16 bit)
rowimm1 = size(image1,1);
colimm1 = size(image1,2);

file_size_MB2 = numel(image2) * 2 + HeadLength; % Ogni pixel
   occupa 2 byte (16 bit)
rowimm2 = size(image2,1);
colimm2 = size(image2,2);

folderPath = 'unite';
CamJA = sprintf('CamJ_00%dA.b16', ind);
CamJB = sprintf('CamJ_00%dB.b16', ind);
% Definisce il percorso completo per il file.b16
filePath1 = fullfile(folderPath, CamJA);
filePath2 = fullfile(folderPath, CamJB);

fileID1 = fopen(filePath1, 'wb');
fileID2 = fopen(filePath2, 'wb');


% Scrive le immagini in un file binario
fwrite(fileID1,'PCO-', 'char');
fwrite(fileID1,file_size_MB1, 'uint32');
fwrite(fileID1,HeadLength, 'uint32');
fwrite(fileID1,rowimm1, 'uint32');
fwrite(fileID1,colimm1, 'uint32');
fwrite(fileID1, 0, 'uint32');
fwrite(fileID1,image1,"uint16");

fwrite(fileID2,'PCO-', 'char');
fwrite(fileID2,file_size_MB2, 'uint32');
fwrite(fileID2,HeadLength, 'uint32');
fwrite(fileID2,rowimm2, 'uint32');
```

```
        fwrite(fileID2,colimm2, 'uint32');
        fwrite(fileID2, 0, 'uint32');
        fwrite(fileID2,image2,"uint16");
        % Chiude i files
        fclose(fileID1);
        fclose(fileID2);
    end
end
```

Script usato per il confronto dei risultati con PIVlab

```
% Script MATLAB realizzato per il Laboratorio 0 ( Misure PIV ) del
   corso di Fluidodinamica Sperimentale , A . A . 2024-25.
% Autori : Federico Poleni , Alberto Vacca , Giacomo Visconti ,
% Marco Zappoli


clear
close all
clc


% Velocit  in galleria

u_gall = 10; %[m/s]

% Caratteristiche piv
pix = 119; %[px]
deltat = 1e-4; %[s]
true_distance = 17.57e-3; %[mm]
M = pix/true_distance; %[px/mm]

% Importa risultati PIV tramite algoritmo di FFT
X = load("X_10ms.mat"); % Spostamento in x in px
Y = load("Y_10ms.mat"); % Spostamento in y in px

% Importa risultati da PIV_lab
Piv_lab = load("Magnitude Velocity 10_ms single step.mat");

%Flippa le matrici perch  le immagini sono riflesse
velocity_magnitude_pivlab = flipud(Piv_lab.velocity_magnitude); %[m/s]
u_velocity_piv = flipud(Piv_lab.u_component); %[m/s]
v_velocity_piv = flipud(Piv_lab.v_component); %[m/s]

% Importo maschere
maschera_fascione = struct2array(load("mascheratura_fascione.mat"));
maschera_nacelle1 = struct2array(load("mascheratura_nacelle1.mat"));
maschera_nacelle2 = struct2array(load("mascheratura_nacelle2.mat"));

% Grandezza figure
[numWindowsY, numWindowsX,~] = size(X.X);

% Media risultati sui 100 frame
X_medio = zeros(numWindowsY, numWindowsX);
Y_medio = zeros(numWindowsY, numWindowsX);
for i = 1 : numWindowsY
    for j = 1 : numWindowsX
        X_medio(i, j) = mean(X.X(i, j, :));
        Y_medio(i, j) = mean(Y.Y(i, j, :));
    end
end
```

```
% Conversione degli spostamenti in velocit
u_velocity = X_medio/(M*deltat); %[m/s]
v_velocity = Y_medio/(M*deltat); %[m/s]

% Applica la maschera sia a PIVlab che al codice implementato dal
   gruppo
for i = 1:numWindowsY
        for j = 1:numWindowsX

            if maschera_fascione((i*16), (j*16)) == 0 ||
               maschera_nacelle1((i*16), (j*16)) == 0 ||
               maschera_nacelle2((i*16), (j*16)) == 0
                u_velocity(i,j) = 0;
                v_velocity(i,j) = 0;
                u_velocity_piv(i,j) = 0;
                v_velocity_piv(i,j) = 0;
                velocity_magnitude_pivlab(i,j) = 0;
            end

        end
end

% Calcolo modulo di velocit  e elimina eventuale vettori spuri da
   entrambe
% le soluzioni
velocity_magnitude = sqrt(u_velocity.^2 + v_velocity.^2); %[m/s]
velocity_magnitude(velocity_magnitude <= 0) = NaN;
velocity_magnitude_pivlab(velocity_magnitude_pivlab <= 0) = NaN;

% Calcolo errore relativo percentuale
diff = abs(velocity_magnitude-velocity_magnitude_pivlab)./u_gall*100;
```